[INSERT: University of Mumbai emblem]

GeoSwipe

Interactive Earth and Indian Heritage Exploration using Hand Gestures

LY Innovation-Based Major Project-B Report

Submitted in partial fulfilment of the requirements of the Degree of

Bachelor of Technology in Computer Engineering

by

Name of Student	**Roll No.**
Kushal Soni	[INSERT: Roll No.]
Kartik Verma	[INSERT: Roll No.]
Bhavana Suthar	[INSERT: Roll No.]
Shreeya Sati	[INSERT: Roll No.]

Supervisor / Guide

Prof. Pradnya Patil

[INSERT: Somaiya Vidyavihar logo]

Department of Computer Engineering

K. J. Somaiya Institute of Technology

An Autonomous Institute permanently affiliated to University of Mumbai

Ayurvihar, Sion, Mumbai-400022

2026-27

Certificate

K. J. Somaiya Institute of Technology, Sion, Mumbai-400022
(An Autonomous Institute permanently affiliated to University of Mumbai)

This is to certify that the project entitled **"GeoSwipe: Interactive Earth and Indian Heritage Exploration using Hand Gestures"** is a bonafide work of **Kushal Soni, Kartik Verma, Bhavana Suthar and Shreeya Sati** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology in Computer Engineering** during the academic year 2026-27.

_____

Prof. Pradnya Patil
Project Guide

_____ _____

Dr. Sarita Ambadekar
Head of Department

Dr. Vivek Sunnapwar
Principal

Place: Sion, Mumbai-400022

Date: **[INSERT: date]**

Project Approval for L.Y.

This project report entitled **"GeoSwipe: Interactive Earth and Indian Heritage Exploration using Hand Gestures"** by the following students is approved for the degree of **Bachelor of Technology in Computer Engineering**.

Name of Student	**Roll No.**
Kushal Soni	[INSERT: Roll No.]
Kartik Verma	[INSERT: Roll No.]
Bhavana Suthar	[INSERT: Roll No.]
Shreeya Sati	[INSERT: Roll No.]

External Examiner	**Internal Examiner**
Name and Signature	Name and Signature

Place: Sion, Mumbai-400022

Date: **[INSERT: date]**

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea, data, fact or source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Name of Student	**Roll No.**	**Signature**
Kushal Soni	[INSERT: Roll No.]	_____
Kartik Verma	[INSERT: Roll No.]	_____
Bhavana Suthar	[INSERT: Roll No.]	_____
Shreeya Sati	[INSERT: Roll No.]	_____

Date: **[INSERT: date]**

Acknowledgement

We take this opportunity to express our sincere gratitude to our project guide, **Prof. Pradnya Patil**, for her continuous guidance, patient reviews and constructive criticism at every stage of this project. Her insistence on grounding every claim in evidence shaped not only this report but also the way we engineered and tested GeoSwipe.

We are thankful to **Dr. Sarita Ambadekar**, Head of the Department of Computer Engineering, for providing the academic environment, laboratory facilities and encouragement that made this work possible, and to **Dr. Vivek Sunnapwar**, Principal, K. J. Somaiya Institute of Technology, for his support of innovation-based projects at the institute.

We also thank the teaching and non-teaching staff of the Department of Computer Engineering for their help with project reviews, infrastructure and administrative matters. Finally, we thank our friends and families, who tested early versions of GeoSwipe, reported the bugs we could not see ourselves, and supported us throughout the academic year.

Kushal Soni	**Kartik Verma**
Bhavana Suthar	Shreeya Sati

Abstract

Geography and cultural heritage are still taught largely through static text, photographs and flat maps, which give learners few opportunities for active exploration. This project presents **GeoSwipe**, a web-based interactive learning platform that unifies three-dimensional (3D) Earth exploration, an interactive map of Indian heritage sites, game-based learning, conversational artificial intelligence (AI) and touch-free hand-gesture control in a single browser application. The front end is a React single-page application that renders a WebGL globe with Three.js and custom GLSL shaders, and a MapLibre GL map of 126 heritage sites in eight categories across 22 states and union territories. Learners open monument panels with live weather and news, read narrated storybooks, play country, flag and heritage quizzes (780 curated questions) alone or in real-time two-player Socket.IO rooms, and query a Groq-hosted large language model (LLM) for heritage answers, trip itineraries and route-safety guidance. Hand gestures are recognised entirely in the browser using MediaPipe hand landmarks, a rule-based classifier for nine gestures and a five-frame stabiliser, so no video frame leaves the device. A Node.js/Express back end keeps all third-party API keys server-side and uses a two-tier (memory and MongoDB) cache with request de-duplication and API-key pooling to operate within free-tier quotas. On the deployed system, cache hits reduced server response times by 20× to 4,363×, a tiered news cache cut NewsAPI calls for a representative four-monument sequence from about 32 to 5, and warm API responses had a median end-to-end latency of 351–2,233 ms from a client in Mumbai. In a synthetic robustness test of 18,000 poses per noise level, gesture accuracy remained at or above 99.06 % up to a landmark noise of $\sigma = 0.01$, and the stabiliser emitted no incorrect gestures in any tested condition. Formal usability and learning-gain studies remain future work.

Keywords: cultural heritage education, 3D geovisualisation, WebGL, hand-gesture recognition, MediaPipe, game-based learning, large language models, real-time multiplayer.

Contents

Certificate i
Project Approval for L.Y. ii
Declaration iii
Acknowledgement iv
Abstract v
List of Figures viii
List of Tables x
List of Abbreviations xii
Chapter 1: Introduction 1
1.1 Introduction 1
1.2 Problem Definition 1
1.3 Aim and Objectives 2
1.4 Organization of the Report 3
Chapter 2: Review of Literature 4
2.1 Literature Survey 4
2.2 Research Gaps 8
Chapter 3: Requirement Specification 10
3.1 Introduction 10
3.2 Hardware Requirements 10
3.3 Software Requirements 11
3.4 Feasibility Study 12
3.5 Cost Estimation 13
3.6 Functional Requirements 14
3.7 Non-Functional Requirements 15
3.8 Project Schedule 16
Chapter 4: Project Analysis and Design 17
4.1 Introduction 17
4.2 UML Diagrams 17
4.3 System Architecture 25
4.4 Database Design 26
4.5 Interface Design 28
Chapter 5: Methodology 30
5.1 Introduction 30
5.2 Development Methodology 30
5.3 System Block Diagram 31
5.4 In-Browser Hand-Gesture Recognition 32

5.5 3D Globe Rendering and Picking 35
5.6 Two-Tier Cache with Request De-duplication 36
5.7 API-Key Pooling 38
5.8 Tiered News Retrieval 39
5.9 Route-Safety Scoring 39
5.10 Multiplayer Synchronisation 40
5.11 AI Services 41
Chapter 6: Implementation Details 43
6.1 Introduction 43
6.2 System Implementation 43
6.3 Front-End Implementation 48
6.4 Back-End Implementation 49
6.5 Security Implementation 50
6.6 Storybook Content Pipeline 50
6.7 Deployment 51
6.8 Testing 51
Chapter 7: Result Analysis 53
7.1 Introduction 53
7.2 Performance Measures 53
7.3 Screenshots 62
7.4 Comparative Analysis 62
7.5 Discussion and Limitations 63
Chapter 8: Conclusion and Future Scope 65
8.1 Conclusion 65
8.2 Future Scope 66
References 67
Published Papers 69
Certificates 70
Plagiarism Report 71

List of Figures

Figure 3.1: Monthly commit activity in the GeoSwipe repository (July 2025 – September 2026) 16
Figure 4.1: Use-case diagram of GeoSwipe 18
Figure 4.2: Sequence diagram for viewing monument details and weather 19
Figure 4.3: Sequence diagram for a two-player multiplayer game 20
Figure 4.4: State diagram of the gesture stabiliser 21
Figure 4.5: State diagram of a multiplayer room 21
Figure 4.6: Level-0 data flow diagram (context diagram) 22
Figure 4.7: Level-1 data flow diagram 22
Figure 4.8: Level-2 data flow diagram of process 5.0 (contextual data proxy and cache) 23
Figure 4.9: Class diagram of GeoSwipe's data models and core services 24
Figure 4.10: Activity diagram of a learning session 25
Figure 4.11: Three-tier architecture of GeoSwipe 26
Figure 4.12: Deployment diagram of the production system 26
Figure 5.1: Iterative and incremental development methodology followed for GeoSwipe 30
Figure 5.2: System block diagram of GeoSwipe 32
Figure 5.3: In-browser gesture-recognition pipeline 32
Figure 5.4: MediaPipe hand landmarks; those read by the classifier are highlighted 33
Figure 5.5: Flowchart of the getOrFetch cache algorithm 36
Figure 5.6: Flowchart of the API-key pool 38
Figure 5.7: Four-tier news retrieval with shared per-tier caching 39
Figure 5.8: Computation of the route-safety and composite scores 40
Figure 6.1: (a) Landing page with mode cards and global toolbar; (b) hand-gesture guide on the landing page 43
Figure 6.2: (a) Explore mode with the interactive 3D globe and rendering controls; (b) flag-guessing game answered by clicking the country on the globe 44
Figure 6.3: (a) Country quiz with multiple-choice options; (b) multiplayer lobby for creating or joining a room 45
Figure 6.4: (a) Heritage map of India with category icons for all sites; (b) searching a site with auto-complete in the Fly-to panel 45
Figure 6.5: (a) Monument sidebar for the Taj Mahal; (b) How-to-Reach page for the selected monument 46
Figure 6.6: (a) Storybook cover for the Taj Mahal; (b) open spread with illustration and narrated chapter text 46

Figure 6.7: (a) Monument-specific heritage quiz question; (b) AI heritage assistant answering a question about the Ajanta Caves.....47
Figure 6.8: (a) AI trip-planner input form; (b) safety-navigation page with route strategy, map and emergency contacts.....48
Figure 7.1: Cold versus warm response times of cached endpoints (logarithmic scale) 54
Figure 7.2: Median and 95th-percentile end-to-end latency of production endpoints 56
Figure 7.3: Per-gesture accuracy (%) at each noise level 58
Figure 7.4: Raw per-frame error versus wrong gestures emitted after stabilisation ... 59
Figure 7.5: Cumulative network data for gesture recognition: server-side streaming versus in-browser inference 60
Figure 7.6: Size of the largest production chunks, raw and compressed.....61
Figure 7.7: Distribution of heritage sites by category and by state or union territory 62

List of Tables

Table 2.1: Summary of the literature survey 6
Table 2.2: Research gaps and GeoSwipe's design response 9
Table 3.1: Client (learner) hardware requirements 10
Table 3.2: Development and test machine (measured) 11
Table 3.3: Front-end software stack 11
Table 3.4: Back-end software stack 11
Table 3.5: External services and free-tier limits 12
Table 3.6: Monthly infrastructure cost for three deployment options 13
Table 3.7: Functional requirements 14
Table 3.8: Non-functional requirements 15
Table 3.9: Project schedule reconstructed from the commit history 16
Table 4.1: MongoDB collections 27
Table 4.2: Structure of the HeritageSite document 27
Table 4.3: REST API endpoints 28
Table 4.4: Socket.IO events 28
Table 5.1: Gesture classification rules (evaluated top to bottom) 33
Table 5.2: Mapping of gestures to interface actions 34
Table 5.3: Cache namespaces and freshness periods 37
Table 5.4: LLM profiles enforced by the server 41
Table 6.1: Principal front-end modules 48
Table 6.2: Back-end modules 49
Table 6.3: Test cases and results 52
Table 7.1: Cold versus warm server response time of cached endpoints (team measurement) 54
Table 7.2: NewsAPI usage for a four-view sequence with tiered caching (team measurement) 54
Table 7.3: End-to-end latency of production API endpoints (report measurement, n = 10 per endpoint) 55
Table 7.4: Classification accuracy versus landmark noise (synthetic benchmark, 2,000 poses per gesture per level) 57
Table 7.5: Stabiliser effect under noise (simulated 30 FPS camera, 300 runs of 3 s per condition) 58
Table 7.6: Timing behaviour of the gesture subsystem (simulation on the development laptop) 59
Table 7.7: Server-side versus in-browser gesture recognition 60

Table 7.8: Capability comparison with systems from the literature survey.....62
Table 7.9: Earlier prototype versus final GeoSwipe.....63

List of Abbreviations

Abbreviation	Full Form	Abbreviation	Full Form
2D / 3D	Two-dimensional / Three-dimensional	JSON	JavaScript Object Notation
AI	Artificial Intelligence	LLM	Large Language Model
API	Application Programming Interface	LOC / KLOC	Lines of Code / Thousand Lines of Code
CDN	Content Delivery Network	MCP / PIP	Metacarpophalangeal / Proximal Interphalangeal (joints)
COCOMO	Constructive Cost Model	NoSQL	Non-relational (“Not only SQL”) database
CORS	Cross-Origin Resource Sharing	OSM	OpenStreetMap
CPU / GPU	Central / Graphics Processing Unit	PDF	Portable Document Format
CSP	Content Security Policy	REST	Representational State Transfer
CSS	Cascading Style Sheets	SPA	Single-Page Application
CVD	Colour Vision Deficiency	SUS	System Usability Scale
DFD	Data Flow Diagram	TTL	Time To Live
FCP	First Contentful Paint	UI / UX	User Interface / User Experience
FPS	Frames Per Second	UML	Unified Modeling Language
GIS	Geographic Information System	UNESCO	United Nations Educational, Scientific and Cultural Organization
GLSL	OpenGL Shading Language	URL	Uniform Resource Locator
HCI	Human–Computer Interaction	UT	Union Territory
HSTS	HTTP Strict Transport Security	WASM	WebAssembly
HTTP(S)	Hypertext Transfer Protocol (Secure)	WebGL	Web Graphics Library
IEEE	Institute of Electrical and Electronics Engineers	WSS	WebSocket Secure
ISO	International Organization for Standardization	XR	Extended Reality

Chapter 1: Introduction

1.1 Introduction

Geography and cultural heritage are knowledge domains in which *where* something is matters as much as *what* it is. A student who reads that the Ajanta Caves were rediscovered in 1819 learns a fact; a student who can spin a globe, zoom into the Sahyadri ranges, open the caves on a map, hear their story narrated chapter by chapter, check today's weather at the site and then test that knowledge in a quiz against a classmate forms a connected mental model of place, history and context. Research on educational games reports that well-designed game mechanics are associated with better learning outcomes, motivation and engagement [1], provided the mechanics are tied to learning goals rather than added as decoration [2]. In parallel, 3D virtual environments and digital reconstructions have been shown to be effective contexts for cultural-heritage learning [3], [4], interaction design has been identified as central to heritage applications in extended reality (XR) [5], [6], and globe-based 3D geographic information systems (GIS) and WebGL have made large geospatial scenes viewable in an ordinary browser [7], [8]. Vision-based hand-gesture recognition offers a natural, touch-free way to manipulate such scenes [9], [10], while large language models (LLMs) and educational chatbots allow learners to ask open-ended questions in natural language [11], [12]. Context-aware itinerary recommendation [13] and multiplayer serious games [14] extend learning from the screen to real-world travel and to learning with peers. Standardised instruments also exist to measure how usable such educational software is [15]. GeoSwipe, the system described in this report, brings these strands together in a single web application that any learner can open from a browser without installing software.

1.2 Problem Definition

Although each of the technologies above has been studied, a learner who wants to explore Indian heritage in its geographic context today must stitch together several disconnected tools: a static textbook or encyclopaedia page for history, a separate map application for location, a weather or news site for current context, a quiz application for practice, and a travel portal for planning a visit. None of these tools share context with one another, most are designed for individual rather than social learning, and all

of them assume a mouse, keyboard or touch screen. From the perspective of a student developer, there is also a practical problem: modern interactive features such as LLM assistants, live weather, news and map tiles depend on third-party application programming interfaces (APIs) whose free tiers are small (for example, 100 NewsAPI requests per day), so a naïve implementation that calls these services on every page view stops working after a handful of visitors. Finally, our own earlier prototype recognised hand gestures in a Python process on the server, which required streaming webcam frames to the server — roughly 900 KB/s per active user — raising both privacy and hosting-cost concerns.

The problem addressed by this project can therefore be stated as follows: **to design, implement and evaluate a single browser-based platform that connects 3D geographic exploration with Indian cultural-heritage content, game-based and multiplayer learning, AI-assisted guidance and touch-free gesture interaction, while protecting user privacy and remaining deployable on free-tier infrastructure.**

1.3 Aim and Objectives

The aim of the project is to build GeoSwipe, an interactive, privacy-preserving and low-cost web platform that makes learning world geography and Indian cultural heritage exploratory, playful and accessible. The specific objectives are:

1. To develop an interactive 3D Earth globe using Three.js and WebGL on which learners can rotate, zoom and select countries, maintaining smooth rendering through adaptive frame-rate control.
2. To build an interactive heritage map of 126 Indian heritage sites, each with descriptive information, access routes, visitor information, multimedia (images, 360° views and 3D models), live weather and related news.
3. To implement game-based learning modules — a country quiz, a flag-guessing game on the globe and monument-specific heritage quizzes — together with real-time two-player multiplayer modes using Socket.IO.
4. To provide narrated, illustrated heritage storybooks that present monument histories as chapters with synchronised audio.

5. To integrate an LLM-based heritage assistant, an AI trip planner that produces structured day-wise itineraries with cost estimates and a downloadable PDF, and a safety-navigation module that ranks routes by a transparent safety score.
6. To enable touch-free interaction through hand gestures recognised entirely in the browser with MediaPipe, including a stabilisation mechanism that suppresses spurious and repeated actions.
7. To secure the system by keeping all API keys on the server, applying security headers, a CORS allow-list, input validation and rate limiting.
8. To make the platform sustainable on free-tier hosting through a shared two-tier cache, request de-duplication and API-key pooling, and to measure the performance of the deployed system quantitatively.

1.4 Organization of the Report

The remainder of this report is organised as follows. **Chapter 2** reviews the literature on digital heritage education, game-based learning, 3D geovisualisation, gesture-based interaction, conversational AI in education, context-aware tourism and multiplayer serious games, and identifies the research gaps that GeoSwipe addresses. **Chapter 3** specifies the hardware and software requirements, analyses technical, economic and operational feasibility, estimates cost and effort, and lists the functional and non-functional requirements. **Chapter 4** presents the analysis and design of the system through Unified Modeling Language (UML) diagrams, data flow diagrams (DFDs), the three-tier architecture, the database schema and the API design. **Chapter 5** describes the methodology: the development process, the system block diagram and the algorithms and mathematical formulations behind gesture recognition, globe picking, caching, key pooling, news retrieval, safety scoring and multiplayer synchronisation. **Chapter 6** documents the implementation with annotated screenshots of every module, deployment details and testing. **Chapter 7** analyses the results — measured latency, cache effectiveness, gesture-recognition robustness, bundle size and page load — and compares GeoSwipe with systems from the literature. **Chapter 8** concludes the report and outlines future work. References and placeholders for published papers, certificates and the plagiarism report follow.

Chapter 2: Review of Literature

2.1 Literature Survey

Search strategy. We searched IEEE Xplore, Google Scholar, MDPI, Springer Link, SAGE and the ISPRS archives using combinations of the keywords *"digital cultural heritage education"*, *"3D virtual environment learning"*, *"virtual globe / WebGL GIS heritage"*, *"educational games learning outcomes"*, *"vision-based hand gesture recognition"*, *"on-device hand tracking"*, *"large language models in education"*, *"educational chatbots"*, *"context-aware itinerary recommendation"*, *"multiplayer serious games"* and *"System Usability Scale"*. We included peer-reviewed journal articles, conference papers, systematic reviews and widely used technical specifications that (a) addressed at least one capability that GeoSwipe implements and (b) described either a working system, an empirical evaluation or a synthesis of prior evidence. Purely promotional material and papers without a method description were excluded. The retained works were grouped into seven themes, discussed below and summarised in Table 2.1.

2.1.1 Game-Based Learning

Yu, Gao and Wang [1] synthesised empirical studies on educational games and reported that games are associated with improvements in learning outcomes, student motivation, engagement and satisfaction, while noting that the size of the effect depends on how the game is designed and used. Abdul Jabbar and Felicia [2] systematically reviewed gameplay engagement in game-based learning and argued that engagement produces learning only when game mechanics are aligned with the learning objectives; features added purely for entertainment can distract. These findings directly informed GeoSwipe's design rule that every game mechanic — clicking a country on the globe to answer a flag question, or choosing between options about a monument the learner has just explored — must exercise geographic or heritage knowledge.

2.1.2 Digital Cultural Heritage and 3D Environments

Mortara and Catalano [3] examined 3D virtual environments as learning contexts for cultural heritage and concluded that they support active exploration that static media cannot provide. Liritzis, Volonakis and Vosinakis [4] reconstructed the Sanctuary of Delphi in 3D and used the reconstruction as an educational approach, showing how

visual reconstructions can make an archaeological site comprehensible. Okanovic *et al.* [5] analysed interaction techniques in extended-reality applications for cultural heritage and emphasised that the quality of interaction, not just visual fidelity, determines the user experience. Belhi *et al.* [6] proposed an integrated framework that couples interaction with 3D visualisation of heritage assets. A common limitation of these works is that each focuses on one site or one collection and relies on specialised content production or XR hardware, whereas a national-scale learning tool needs to cover many sites on commodity devices.

2.1.3 3D Geovisualisation and Virtual Globes

Scianna and La Guardia [7] demonstrated globe-based 3D GIS solutions for virtual heritage, placing heritage models at their true geographic coordinates on a virtual globe. Wüest *et al.* [8] described the processing and rendering of massive 3D geospatial environments using WebGL in the OpenWebGlobe and SmartMobileMapping projects, establishing that browser-based rendering of global-scale scenes is feasible. These works motivate GeoSwipe's use of a WebGL globe as the entry point to exploration, but they target GIS professionals and do not include learning activities, gamification or natural interaction.

2.1.4 Vision-Based Hand-Gesture Interaction

Zabulis, Baltzakis and Argyros [9] reviewed vision-based hand-gesture recognition for human–computer interaction (HCI), covering hand detection, tracking and gesture classification. Kölsch and Turk [10] built vision-based hand-gesture interfaces for wearable computing and virtual environments and highlighted robustness to lighting and background as a practical challenge. More recent work on on-device perception changed what is practical in a browser: MediaPipe Hands [16] combines a palm detector with a landmark model that predicts 21 3D hand keypoints in real time on commodity hardware, and it is built on the MediaPipe framework [17], which models perception as a graph of reusable calculators. GeoSwipe builds on this line of work: it uses MediaPipe's hand-landmark model compiled to WebAssembly (WASM) inside the browser and adds a lightweight geometric classifier and stabiliser on top of the landmarks.

2.1.5 AI-Assisted Learning and Conversational Agents

Dong *et al.* [11] systematically reviewed the use of large language models in education, identifying question answering, personalised explanation and content generation as major uses, together with concerns about accuracy and over-reliance. Bekkar and Chtouki [12] reviewed chatbots in education and reported that they are used for learner support and information access, but that many deployments lack rigorous evaluation. GeoSwipe's heritage assistant follows these recommendations by grounding the LLM with the names and categories of the heritage sites stored in the platform's own database and by fixing generation parameters on the server.

2.1.6 Context-Aware Tourism

Fogli and Sansonetti [13] exploited semantic information to recommend context-aware itineraries that account for points of interest, user preferences and contextual factors. This demonstrates that exploration can move from learning about a place to planning a visit, a transition that GeoSwipe supports through its AI trip planner and its safety-navigation module, both of which use the selected monument, its visitor information and live weather as context.

2.1.7 Multiplayer Learning and Usability Evaluation

Wendel *et al.* [14] studied collaborative learning in multiplayer serious games and showed that shared game spaces add a social dimension to learning. GeoSwipe's two-player rooms for flag, country and heritage quizzes apply the competitive form of this idea. For evaluation, Brooke's System Usability Scale (SUS) [18] is a ten-item questionnaire widely used to measure perceived usability, and Orfanou, Tselios and Katsanos [15] empirically evaluated SUS on learning-management systems and found it suitable for educational software. We use these works to define the planned user study in Chapter 8.

Table 2.1: Summary of the literature survey

Author(s) / Year	Method / Contribution	Dataset / Context	Results / Findings	Limitations
Yu, Gao, Wang (2021) [1]	Synthesis of empirical studies on educational games	Published studies on educational games	Games associated with better learning outcomes, motivation, engagement and satisfaction	Effect depends on design; not specific to geography or heritage

Author(s) / Year	Method / Contribution	Dataset / Context	Results / Findings	Limitations
Abdul Jabbar, Felicia (2015) [2]	Systematic review of gameplay engagement	Game-based learning studies	Engagement supports learning when mechanics align with objectives	Heterogeneous studies; engagement is not learning by itself
Mortara, Catalano (2018) [3]	Analysis of 3D virtual environments for heritage learning	Cultural-heritage virtual environments	3D environments enable active, exploratory learning	Content production cost; single-site focus
Liritzis et al. (2021) [4]	3D reconstruction used as an educational approach	Sanctuary of Delphi	Reconstruction makes the site comprehensible to learners	One site; heavy modelling effort
Okanovic et al. (2022) [5]	Study of interaction in XR heritage applications	XR cultural-heritage applications	Interaction quality is central to user experience	Requires XR hardware
Belhi et al. (2024) [6]	Integrated framework for interaction and 3D visualisation	Cultural-heritage assets	Couples interaction with 3D heritage visualisation	Framework-level; no gamified learning or multiplayer
Scianna, La Guardia (2018) [7]	Globe-based 3D GIS for virtual heritage	Georeferenced heritage models	Heritage data can be placed on a virtual globe	GIS-expert oriented; no learning activities
Wüest et al. (2012) [8]	WebGL processing and rendering of massive 3D geodata	OpenWebGlobe, SmartMobileMapping	Browser rendering of global 3D scenes is feasible	Engineering focus; no education features
Zabulis et al. (2009) [9]	Review of vision-based hand-gesture recognition	Surveyed HCI systems	Detection–tracking–recognition pipeline characterised	Predates on-device neural landmark models
Kölsch, Turk (2004) [10]	Vision-based gesture interfaces for wearables and VEs	Wearable / virtual-environment use	Hand gestures usable as natural input	Sensitive to lighting and background
Dong et al. (2024) [11]	Systematic review of LLMs in education	Studies on LLM use in education	QA, explanation and content generation are key uses	Accuracy and over-reliance concerns
Bekkar, Chtouki (2024) [12]	Systematic review of educational chatbots	Chatbot deployments in education	Chatbots support learners and information access	Many deployments lack rigorous evaluation
Fogli, Sansonetti (2019) [13]	Semantic, context-aware itinerary recommendation	Points of interest and context data	Semantics improve itinerary relevance	Not linked to heritage learning
Wendel et al. (2010) [14]	Multiplayer serious games for collaborative learning	Multiplayer serious-game prototype	Shared game spaces add a social learning dimension	Early prototype; limited generalisation
Orfanou et al. (2015) [15]	Empirical evaluation of SUS on LMSs	Learning-management systems and their users	SUS suitable for perceived usability of educational software	Usability only; not learning gain

Author(s) / Year	Method / Contribution	Dataset / Context	Results / Findings	Limitations
Zhang et al. (2020) [16]	MediaPipe Hands: palm detector + landmark model	Hand images; real-time on-device tracking	21 3D keypoints per hand in real time on commodity devices	Landmarks only; gesture semantics left to applications
Lugaresi et al. (2019) [17]	MediaPipe perception-pipeline framework	Cross-platform perception pipelines	Graph-based, reusable, real-time pipelines	Framework, not an end-user application
Brooke (1996) [18]	System Usability Scale (10-item questionnaire)	Industrial usability evaluation	Quick, reliable perceived-usability score	Does not diagnose specific problems
Boehm (1981) [19]	COCOMO software cost-estimation model	Historical software projects	Effort and schedule estimated from size (KLOC)	Calibrated on older, non-framework projects
Fette, Melnikov (2011) [20]	WebSocket protocol specification (RFC 6455)	Internet standard	Full-duplex messaging over a single TCP connection	Protocol only; application semantics left to users
Sinnott (1984) [21]	Haversine formula for great-circle distance	Spherical-Earth geometry	Numerically stable distance for small separations	Spherical approximation of the Earth

2.2 Research Gaps

Reading the literature against GeoSwipe's problem statement reveals the following gaps. Table 2.2 maps each gap to the design response adopted in this project.

1. **Fragmentation of capabilities.** Heritage visualisation [3]–[6], geovisualisation [7], [8], gesture interaction [9], [10], conversational AI [11], [12], tourism recommendation [13] and multiplayer learning [14] are studied separately. We did not find, among the surveyed works, a single system that combines all of them around one geographic learning workflow.
2. **Specialised hardware and content.** Many heritage systems rely on XR headsets or bespoke 3D reconstructions of one site [4], [5], which limits reach to learners with ordinary laptops and phones.
3. **Privacy and cost of gesture recognition.** Classical and server-side gesture pipelines [9], [10] require video to be processed away from the user or on dedicated machines; streaming webcam frames to a server is both a privacy risk and a scaling cost.

4. **Operational sustainability.** Research prototypes rarely address the quota limits and key exposure of third-party APIs, which in practice decide whether a student-built platform can stay online.
5. **Weak coupling between game mechanics and content.** Reviews [1], [2] warn that gamification must be tied to learning goals; many applications add points and badges without spatial or cultural content.
6. **Limited quantitative evaluation.** Chatbot and gamified systems are often reported without measurement [12]; performance, robustness and usability evidence is needed [15], [18].

Table 2.2: Research gaps and GeoSwipe's design response

Gap	GeoSwipe response
Fragmented capabilities	One SPA links globe → heritage map → monument panel → storybook / quiz / AI chat / trip plan / safety route, sharing the selected-site context
Specialised hardware	Runs in a standard browser with WebGL and an ordinary webcam; no headset or installation
Gesture privacy and cost	MediaPipe landmarks computed in the browser; frames never leave the device; zero server compute for gestures
Operational sustainability	Server-side keys, two-tier shared cache, in-flight de-duplication, key pooling with soft caps, stale-on-failure serving
Weak mechanic–content coupling	Answers are given on the globe (flag game) or about the monument just explored (heritage quiz); storybooks precede quizzes
Limited evaluation	Latency, cache, bundle, page-load and gesture-robustness measurements reported in Chapter 7; SUS study planned

Chapter 3: Requirement Specification

3.1 Introduction

This chapter specifies what GeoSwipe needs in order to be built, deployed and used. We first list the hardware and software required by end users, by the development team and by the hosting environment. We then examine whether the project is technically, economically and operationally feasible, estimate its cost and effort, and record the functional and non-functional requirements that guided the design in Chapter 4. Where a value is a measured property of our own environment (for example, the development laptop or the size of the code base) it is reported as measured; where a value is a recommended minimum that we did not test exhaustively, it is marked **[ASSUMED]**.

3.2 Hardware Requirements

GeoSwipe is a web application, so its hardware requirements fall into three groups: the learner's device, the development and test machine, and the hosting infrastructure. The learner's device must be able to render a WebGL globe and, optionally, run the MediaPipe hand-landmark model; both run on the device's graphics processing unit (GPU) when available. Table 3.1 lists the client requirements and Table 3.2 the machine on which GeoSwipe was developed and all client-side measurements in Chapter 7 were taken. The hosting resources are free tiers: Cloudflare Pages serves the static front end from its global content delivery network (CDN), a Render web service in the Singapore region (512 MB RAM, sleeping after 15 minutes of inactivity) runs the back end, and a MongoDB Atlas M0 cluster (512 MB storage) holds the data.

Table 3.1: Client (learner) hardware requirements

Component	Minimum [ASSUMED]	Recommended
Processor	Dual-core x86-64 or ARM64 CPU	Quad-core or better
Memory (RAM)	4 GB	8 GB or more
Graphics	WebGL 2-capable integrated GPU	Discrete or recent integrated GPU
Camera (optional)	Any 640×480 webcam, needed only for gestures	720p webcam, 30 FPS
Display	1280 × 720	1920 × 1080
Network	Broadband connection; ≈ 10 MB extra on first gesture use	Stable broadband

Table 3.2: Development and test machine (measured)

Component	Specification
Processor	13th Gen Intel Core i5-13450HX, 10 cores / 16 threads
Memory	16 GB (15.7 GB usable)
Graphics	NVIDIA GeForce RTX 3050 6 GB Laptop GPU; Intel UHD Graphics
Operating system	Microsoft Windows 11 Home Single Language
Browser	Google Chrome 154
Runtime	Node.js 22.16.0, Python 3.11.5 (captioning script only)

3.3 Software Requirements

Table 3.3 and Table 3.4 list the principal software components with the versions declared in the project's package.json files. The front end is written in JavaScript (JSX) and built with Vite; the back end is a Node.js service. Table 3.5 lists the external services that GeoSwipe integrates, together with the free-tier limits that shaped the caching design in Chapter 5.

Table 3.3: Front-end software stack

Component	Version	Purpose in GeoSwipe
React / React DOM	19.1	Component-based user interface
React Router DOM	7.9	Client-side routing for 18 routes
Vite	7.0	Development server and production bundler
Three.js / three-globe	0.179 / 2.44	WebGL 3D Earth, country borders and picking
MapLibre GL JS	5.7	Vector/raster heritage map with custom icon layers
@mediapipe/tasks-vision	1.0	In-browser hand-landmark detection (WASM, GPU delegate)
Socket.IO client	4.8	Real-time multiplayer communication
Framer Motion	12.38	UI animation
react-pageflip	2.0	Page-turn storybook reader
jsPDF	4.2	Export of AI trip plans to PDF
dat.GUI	0.7.9	Globe lighting, material and animation controls
ESLint / TypeScript	9.30 / 5.6	Linting and type checking

Table 3.4: Back-end software stack

Component	Version	Purpose in GeoSwipe
Node.js	≥ 18	JavaScript runtime
Express	5.1	REST API and proxy routes
Socket.IO	4.8	Multiplayer rooms and event broadcasting
Mongoose	8.17	MongoDB object modelling, indexes and queries

Component	Version	Purpose in GeoSwipe
express-rate-limit	7.1	Per-IP request limiting per route group
compression / cors / dotenv	1.7 / 2.8 / 17.4	gzip responses, CORS allow-list, configuration
Python + OpenAI Whisper	3.11	Offline caption generation for storybook audio (audio-to-captions.py)

Table 3.5: External services and free-tier limits

Service	Used for	Free-tier limit (as documented by the team, Sept. 2026)
Groq	LLM for chatbot, trip planner, safety guidance	Rate-limited free tier
MapTiler	Map styles, tiles, sprites, glyphs	100,000 requests / month
OpenWeatherMap	Current weather and 5-day forecast	1,000 calls / day
NewsAPI	News about sites and safety alerts	100 requests / day
Unsplash / Wikipedia	Monument images (Wikipedia as fallback)	50 requests / hour (Unsplash)
Overpass API (OSM)	Nearby hospitals, police and fire stations	Public instance, IP-throttled
flagcdn.com	Country flag images	Public CDN
MongoDB Atlas	Persistent data and cache	512 MB storage (M0)

3.4 Feasibility Study

3.4.1 Technical Feasibility

Every capability of GeoSwipe maps onto a mature, open-source library that runs in a standard browser or on Node.js. WebGL rendering of a textured globe with custom shaders is well supported by Three.js; MapLibre GL renders thousands of map features with GPU acceleration; and MediaPipe's hand-landmark model is distributed as a WebAssembly runtime that executes on the GPU delegate inside the browser [16], [17]. Real-time multiplayer only requires a persistent WebSocket connection, which Socket.IO provides on top of the WebSocket protocol [20]. The riskiest technical element was gesture recognition. Our first version ran MediaPipe in a server-side Python process; moving it into the browser removed the dependency on server compute entirely, and we verified the port by running the original Python classifier and the new JavaScript classifier on 20,000 generated hand poses, obtaining zero mismatches. The deployed system passing 9 of 10 integration health checks (Chapter 7) confirms technical feasibility end to end.

3.4.2 Economic Feasibility

GeoSwipe was built with free and open-source software and is hosted entirely on free tiers (Cloudflare Pages, Render and MongoDB Atlas M0). The only scarce resources are third-party API quotas. Because weather, news and images are requested for a fixed set of 126 heritage sites, caching their results per site makes API consumption depend on the number of sites and the refresh interval rather than on the number of users; this is what makes a ₹0 monthly operating cost realistic for a showcase deployment. Section 3.5 quantifies the options for scaling beyond the free tiers.

3.4.3 Operational Feasibility

Learners need nothing more than a browser; there are no accounts, installations or plug-ins, and the webcam is optional and used only after explicit permission. Every gesture action also has a mouse or touch equivalent, so the platform remains usable when a camera is unavailable. Teachers can demonstrate the globe and storybooks on a projector and run two-player quiz rounds in class with a six-character room code. For maintainers, a /api/diagnostics endpoint reports the health of each integration with a specific fix hint, and a Render blueprint (render.yaml) and a step-by-step deployment guide make redeployment repeatable.

3.5 Cost Estimation

3.5.1 Infrastructure Cost

Table 3.6 summarises three deployment options analysed by the team from the providers' published pricing (exchange rate assumption: 1 USD = ₹83.5). The current deployment corresponds to Option A.

Table 3.6: Monthly infrastructure cost for three deployment options

Option	Configuration	Monthly cost
A. Showcase (current)	Cloudflare Pages + Render free + Atlas M0 + free API tiers with caching and key pools	₹0
B. Low-cost production	Static front end free; ~1 GB VPS back end; Atlas M0; free map tier; free alternatives for news and weather	≈ ₹500 – ₹1,000
C. Scalable (10,000+ users)	Paid front-end hosting, dedicated server, Atlas M10, MapTiler paid tier, pay-as-you-go LLM	≈ ₹14,000+

The one-time hardware cost is nil because the project was developed on the students' existing laptops. A custom domain, if desired, costs approximately ₹1,000 per year.

3.5.2 Effort Estimation using COCOMO

To estimate development effort we applied the basic Constructive Cost Model (COCOMO) [19] to the size of the code base, which we measured directly. Excluding data and seed files (12,429 lines) and stylesheets (7,480 lines), the source code contains 21,040 lines of client JavaScript/JSX, 4,734 lines of server JavaScript, 33 lines of GLSL and 197 lines of Python, i.e. **KLOC ≈ 26.0**. Treating GeoSwipe as an *organic* project (a small team working on a familiar kind of software):

$$E = a \cdot (KLOC)^b = 2.4 \times (26.0)^{1.05} \approx 73.5 \text{ person-months} \quad (3.1)$$

$$D = c \cdot E^d = 2.5 \times (73.5)^{0.38} \approx 12.8 \text{ months} \quad (3.2)$$

$$P = E / D \approx 5.7 \text{ persons} \quad (3.3)$$

The actual project was delivered by four students working part-time over about fourteen months (July 2025 – September 2026). COCOMO's estimate is higher than the effort actually spent because the model was calibrated on projects written without today's high-level frameworks; much of GeoSwipe's functionality is obtained by composing libraries such as Three.js, MapLibre and MediaPipe rather than writing it from scratch. The estimate is nevertheless useful as an upper bound on the effort that an equivalent system would require without such reuse.

3.6 Functional Requirements

Table 3.7 lists the functional requirements. Each requirement is traceable to a module described in Chapters 5 and 6.

Table 3.7: Functional requirements

ID	Requirement
FR1	The system shall render an interactive 3D Earth that can be rotated, zoomed and clicked to select a country.
FR2	The system shall display 126 Indian heritage sites on a map with category-specific icons and multiple map styles.
FR3	The system shall show, for a selected site, its summary, history, architecture, significance, access routes, visitor information, images, 360° view and 3D model where available.
FR4	The system shall show current weather, a 5-day forecast and recent news for a selected site.
FR5	The system shall provide a country quiz and a flag-guessing game answered on the globe, with difficulty levels.
FR6	The system shall provide monument-specific and all-India heritage quizzes at easy, medium and hard levels.

ID	Requirement
FR7	The system shall allow two players to create or join a room and play 10 synchronised rounds with live scores and a winner.
FR8	The system shall present narrated, illustrated storybooks with page-turn navigation for selected monuments.
FR9	The system shall answer natural-language questions about Indian heritage through an AI assistant.
FR10	The system shall generate a day-wise trip itinerary with cost breakdown and booking links, exportable as PDF.
FR11	The system shall suggest routes ranked by safety, list nearby safe places and show emergency contacts.
FR12	The system shall allow users to move a cursor, click controls and rotate/zoom the globe using hand gestures.
FR13	The system shall provide a welcome guide and light/dark themes.
FR14	The system shall expose health and diagnostics endpoints for maintainers.

3.7 Non-Functional Requirements

Table 3.8: Non-functional requirements

Attribute	Requirement and how it is met
Privacy	Webcam frames must never leave the device — gestures are inferred in the browser; no user accounts or personal data are stored.
Security	API keys must stay server-side; responses carry security headers (CSP, X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy, HSTS in production); CORS allow-list; input escaping against NoSQL regex injection.
Performance	Cached API responses should be served in a few milliseconds server-side; gesture inference is capped at 20 FPS to leave the main thread for rendering.
Availability	Upstream failures must degrade gracefully — stale cached data is served when a provider fails or a quota is exhausted.
Scalability	Third-party API usage must scale with the number of sites, not the number of users; at most 500 concurrent multiplayer rooms are tracked.
Abuse resistance	Rate limits: 1,200 requests / 15 min per IP overall, 20 AI requests / 5 min, 60 weather requests / 5 min, 3,000 map requests / 5 min; 30 room actions / 10 s per socket.
Usability	Every gesture action has a mouse/touch equivalent; on-screen guides explain gestures.
Maintainability	Single source of truth for the API base URL; modular services; lint-clean build.
Portability	Runs in current desktop browsers without installation.

3.8 Project Schedule

The project was developed iteratively between July 2025 and September 2026. Figure 3.1 shows the monthly commit activity in the project's Git repository (90 commits), and Table 3.9 relates the bursts of activity to the features delivered, as recorded in the commit history.

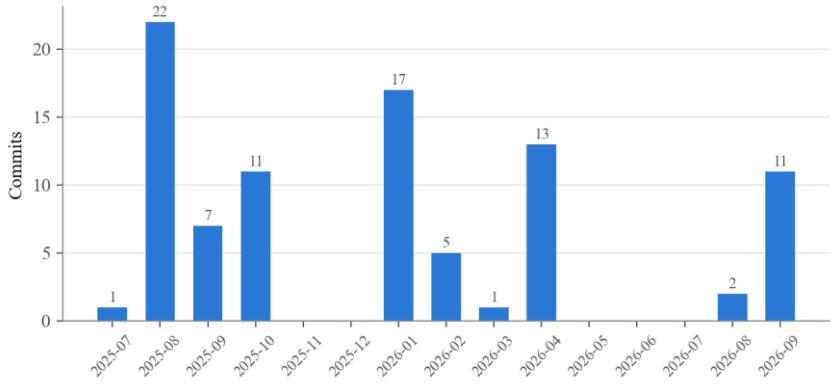


Figure 3.1: Monthly commit activity in the GeoSwipe repository (July 2025 – September 2026)

Table 3.9: Project schedule reconstructed from the commit history

Period	Work completed
Jul – Aug 2025	Repository set-up; gesture-controlled 3D globe; quiz logic and page routing; hand-gesture selection
Sep – Oct 2025	Heritage map (Maharashtra first), monument sidebar, 3D models, information tabs, 360° view, heritage seed data, performance optimisation, first storybook
Jan 2026	Public release of repository; new map styles; gesture click on all pages; control guide, about and contact sections; difficulty levels; more sites
Feb 2026	Browser-camera capture and camera show/hide
Mar – Apr 2026	Flag game; multiplayer quiz, flag and heritage modes; AI chatbot; weather and news; trip planner; safety navigator; Mapathon submission
Aug 2026	Analysis reports and research paper
Sep 2026	Security hardening, server-side API proxies, in-browser gesture recognition, caching and key pools, production deployment

Chapter 4: Project Analysis and Design

4.1 Introduction

This chapter translates the requirements of Chapter 3 into a design. We model the system's behaviour with UML diagrams — use-case, sequence, state, class and activity diagrams — and its data movement with data flow diagrams (DFDs) at levels 0, 1 and 2. We then present the three-tier architecture and its deployment, the MongoDB schema, and the REST and Socket.IO interfaces. The diagrams were drawn from the implemented code, so every element shown corresponds to a real module, route or collection.

4.2 UML Diagrams

4.2.1 Use-Case Diagram

Figure 4.1 shows the use-case diagram. The primary actor is the **Learner / Visitor**, who can explore the 3D globe, select countries, play single-player and multiplayer games, browse the heritage map, view monument details, weather and news, read storybooks, take heritage quizzes, ask the AI assistant, generate and export trip plans, plan safe routes and control the interface with hand gestures. The **Opponent Player** participates only in the multiplayer use case. Three secondary actors provide services: the **Groq LLM service** (chat, trip planning and safety guidance), the **weather, news, map and image APIs**, and the **webcam**, which is drawn as a separate actor to emphasise that it is a device-local source of input. *View Monument Details* includes *View Weather & News*, because the monument panel always requests the site's current conditions.

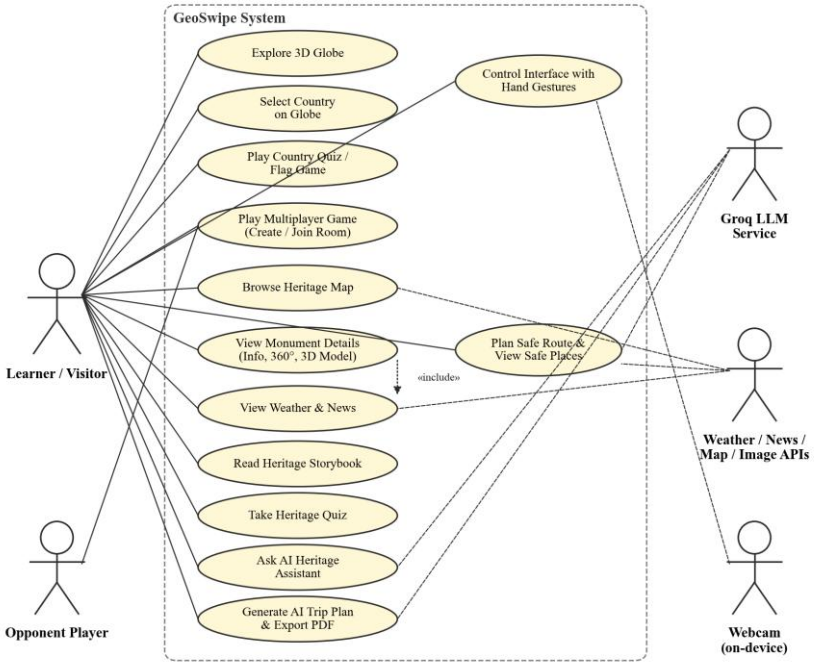


Figure 4.1: Use-case diagram of GeoSwipe

4.2.2 Sequence Diagrams

Figure 4.2 traces the most frequent interaction: selecting a monument on the heritage map. The React client first requests the site's details from the Express API, which looks the site up in MongoDB with an escaped, anchored regular expression. The client then requests current weather. The server calls getOrFetch on the two-tier cache; only if the entry is missing or expired (the *alt* fragment) does the cache call the upstream provider with a key drawn from the key pool, rotating to the next key on an HTTP 429 response, and persist the result to MongoDB. Whether the payload is fresh or a stale fallback, the client receives JSON and renders the sidebar panels.

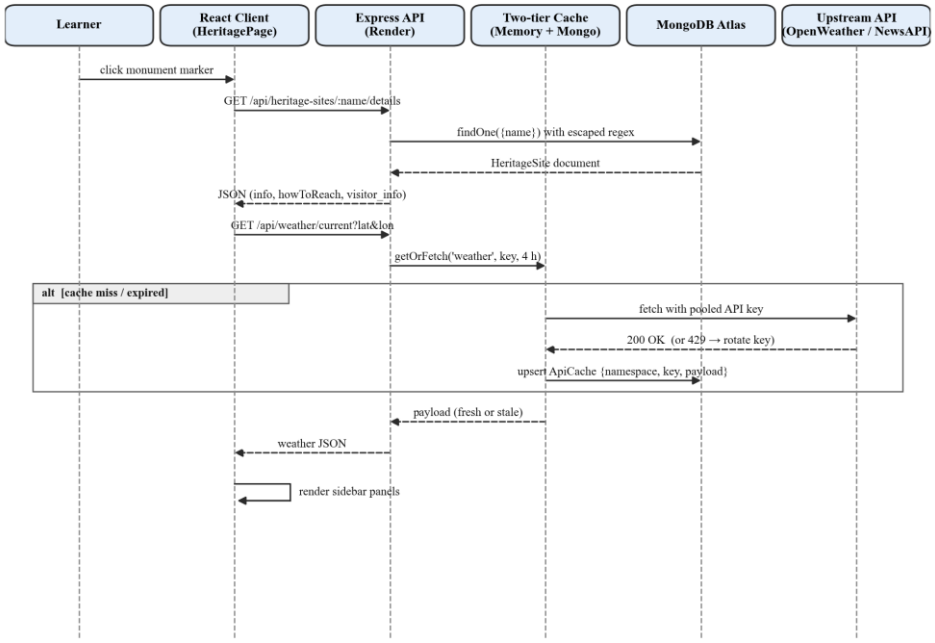


Figure 4.2: Sequence diagram for viewing monument details and weather

Figure 4.3 shows a two-player game. Player 1 sends join-room and the server creates the room, subject to a ceiling of 500 active rooms. When Player 2 joins, the server generates the first question — from the quiz_questions collection for heritage modes or from the country list and flagcdn for the flag mode — and broadcasts game-started and new-question to both players. In each of the ten rounds (*loop* fragment) the server records each player's answer once, announces player-answered without revealing correctness, and, when both have answered, broadcasts show-result with the correct answer and updated scores. After the tenth round it broadcasts game-over with the winner or a tie.

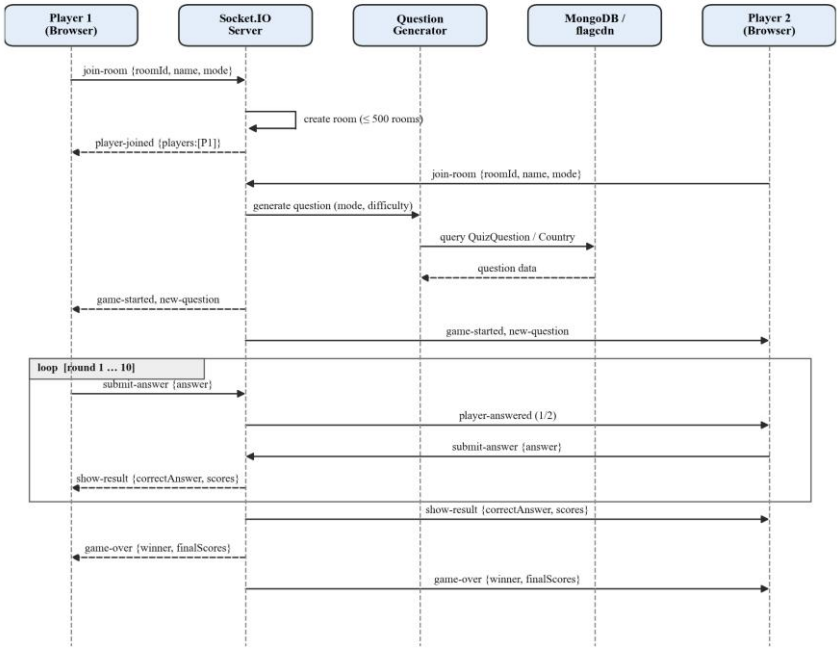


Figure 4.3: Sequence diagram for a two-player multiplayer game

4.2.3 State Diagrams

Figure 4.4 models the gesture stabiliser that sits between the landmark classifier and the user interface. With no hand in view the system is in *No Hand* and the cursor is cleared. When a hand appears, the classified pose becomes a *Candidate* with a count of one; each subsequent frame with the same pose increments the count, and any change of pose resets it. Once the count reaches five and the pose is not *unknown*, the gesture is *Stable* and is emitted on the gesture bus. A *click* is emitted only if at least 1.0 s has passed since the previous click, otherwise it is *Suppressed*; this prevents a held “OK” sign from pressing a button repeatedly. Frames that arrive less than 50 ms after the last processed frame are *Skipped*, capping inference at 20 FPS.

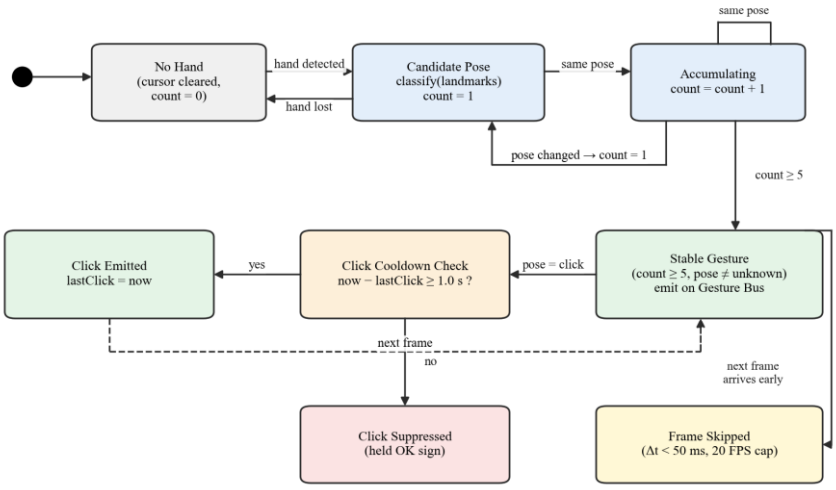


Figure 4.4: State diagram of the gesture stabiliser

Figure 4.5 models the life cycle of a multiplayer room. A room is *Waiting* with one player, moves to *Playing* when the second player joins, alternates between *Collecting Answers* and *Round Result* for each round, and enters *Game Over* after round ten, after which scores are reset and the room returns to *Waiting* for a rematch. A room is deleted when its players leave or when it has been inactive for more than 10 minutes; the server checks for such rooms every minute.

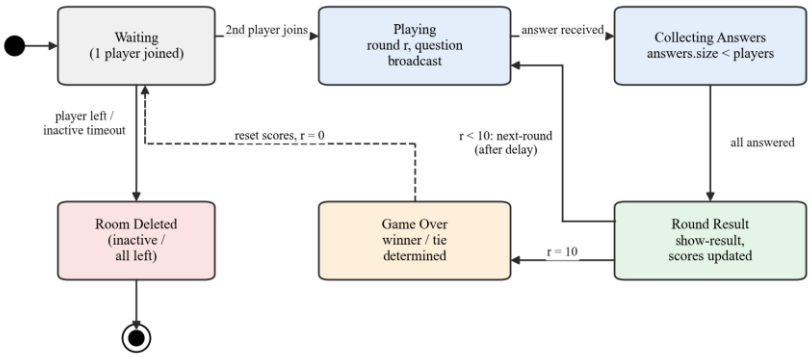


Figure 4.5: State diagram of a multiplayer room

4.2.4 Data Flow Diagrams

The level-0 DFD (context diagram) in Figure 4.6 treats GeoSwipe as a single process. The learner supplies clicks, gestures, queries and trip inputs and receives rendered globes, maps, quizzes, answers and itineraries; the opponent exchanges answers, questions and scores. Video frames flow from the webcam into the system but, as the label states, are processed on the device. Four groups of external entities exchange data

with the system: the Groq LLM, weather/news/map-tile APIs, image APIs and the Overpass OpenStreetMap (OSM) API.

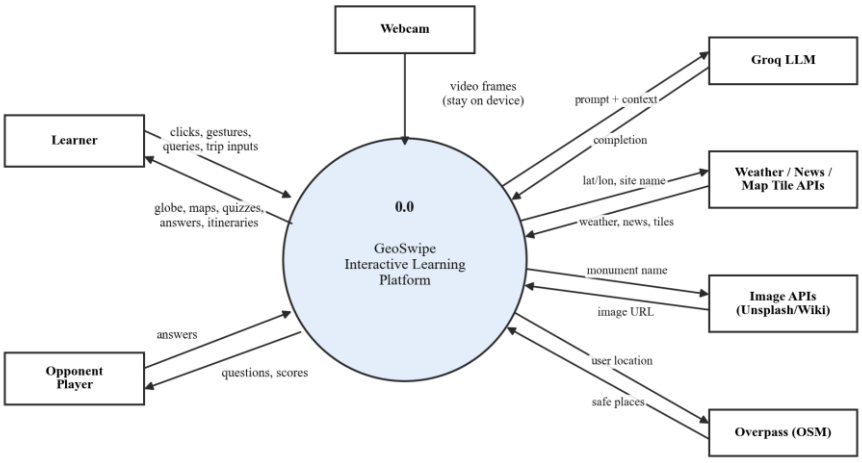


Figure 4.6: Level-0 data flow diagram (context diagram)

The level-1 DFD in Figure 4.7 decomposes the system into six processes: 1.0 Gesture Recognition, 2.0 Globe & Geography Games, 3.0 Heritage Exploration, 4.0 Multiplayer Session Management, 5.0 Contextual Data Proxy & Cache and 6.0 AI Services. The data stores are D1 Heritage Sites, D2 Quiz Questions, D3 Countries, D4 ApiCache (all in MongoDB) and D5 Rooms, which is held in the server's memory. Gesture events from process 1.0 drive both the globe and the heritage map, and process 5.0 is the only process that talks to external data APIs.

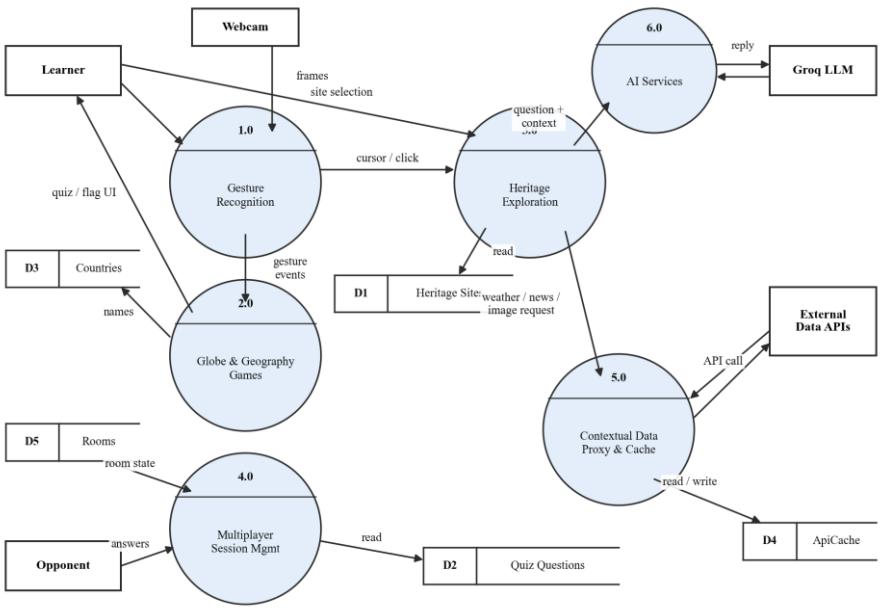


Figure 4.7: Level-1 data flow diagram

The level-2 DFD refines the process that carries the project's main operational contribution. Figure 4.8 decomposes process 5.0: a request is validated and rate-limited (5.1), looked up in the memory map (5.2) and then in the MongoDB ApiCache collection (5.3); on a miss, concurrent requests for the same key are collapsed (5.4), a key is selected from the pool (5.5), the upstream result is stored in both tiers (5.6) and a fresh or stale payload is served (5.7). Process 1.0 is decomposed in the same way in Chapter 5, where the gesture pipeline (capture, landmark detection, mirroring and classification, stabilisation and publication on the gesture bus) is described step by step.

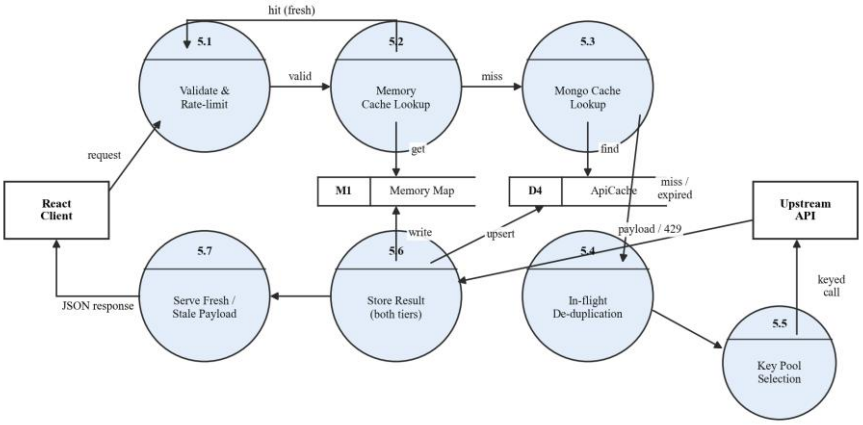


Figure 4.8: Level-2 data flow diagram of process 5.0 (contextual data proxy and cache)

4.2.5 Class Diagram

Although GeoSwipe is written in JavaScript rather than a class-based language, its persistent data and core services have well-defined structure, shown in Figure 4.9. The six Mongoose models (blue) are HeritageSite, QuizQuestion, MonumentImage, Country, FlagImage and ApiCache. HeritageSite carries a 2dsphere geospatial index on its coordinates; ApiCache has a unique compound index on (namespace, key) and a time-to-live (TTL) index of seven days on cachedAt. The service objects (green) are the CacheService, which persists to ApiCache, and the KeyPool, which the cache's fetch functions use. The client-side and in-memory structures (orange) are the GestureStabilizer, which emits events through the GestureBus, and the server's in-memory MultiplayerRoom.

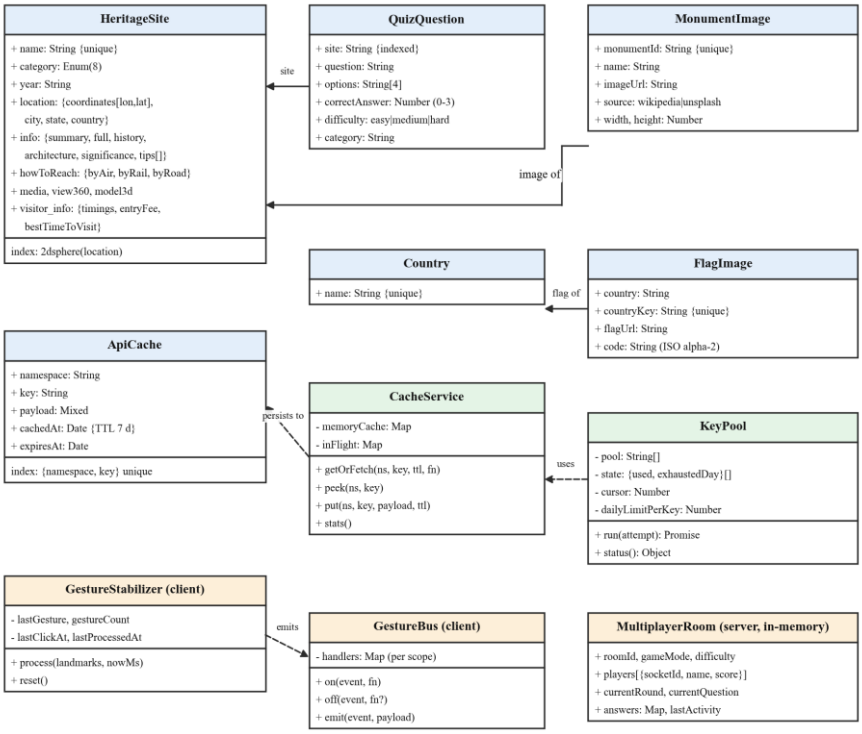


Figure 4.9: Class diagram of GeoSwipe's data models and core services

4.2.6 Activity Diagram

Figure 4.10 shows a typical learning session. From the landing page the learner chooses *Explore* or *Heritage*. In Explore mode, the learner manipulates the globe with a mouse or gestures and may start a single-player quiz or flag game, or create or join a two-player room that runs ten synchronised rounds. In Heritage mode, selecting a monument opens its sidebar, from which several activities can proceed in parallel (fork and join bars): weather and news, the storybook, the heritage quiz and the AI chat or trip plan. The learner may then plan a safe route or export the trip PDF, and either continue exploring or end the session.

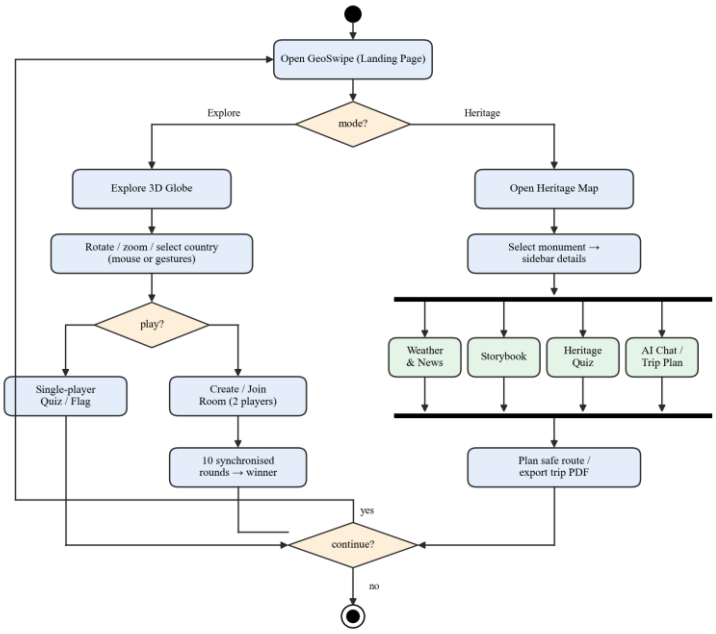


Figure 4.10: Activity diagram of a learning session

4.3 System Architecture

GeoSwipe follows a three-tier architecture, shown in Figure 4.11. The **presentation tier** is a React 19 single-page application (SPA) built with Vite and served as static files from Cloudflare Pages. It contains the landing and navigation shell, the Three.js globe, the MapLibre heritage map, game and multiplayer screens, the storybook reader, the trip planner and safety navigator, and the chatbot widget. The gesture subsystem also lives in this tier. The **application tier** is a Node.js/Express 5 server with Socket.IO, hosted on Render. It applies security headers and a CORS allow-list, enforces rate limits, serves REST routes for heritage, quiz, flag and safety data, proxies every keyed third-party API, manages multiplayer rooms, and implements the two-tier cache and the key pools. The **data tier** is a MongoDB Atlas database holding the heritage sites, quiz questions, countries and cached API payloads. External services are reached only from the application tier, so no API key is ever present in the browser bundle.

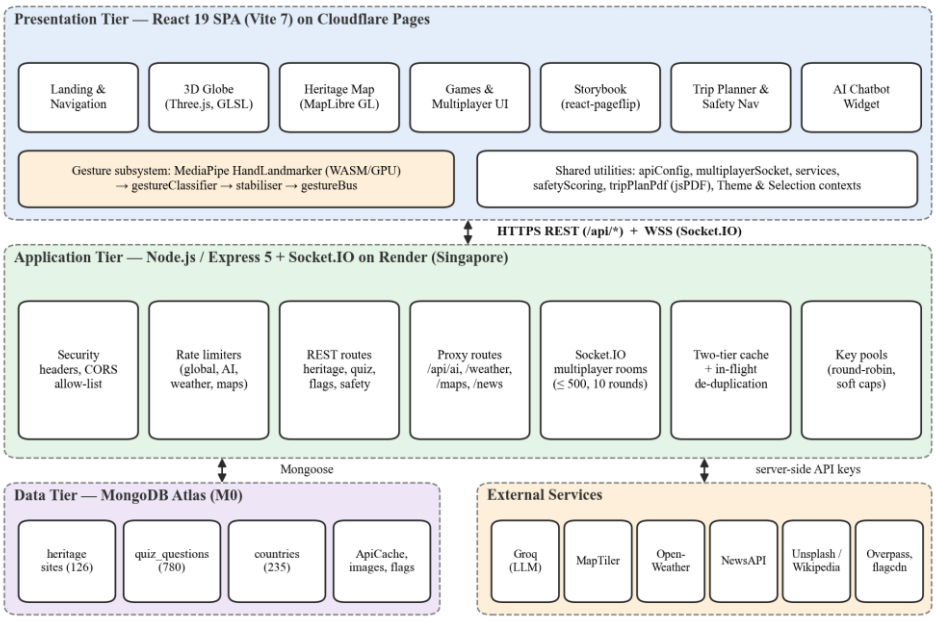


Figure 4.11: Three-tier architecture of GeoSwipe

Figure 4.12 presents the deployment view. The user's browser downloads the static bundle, the MediaPipe WASM runtime and the hand-landmark model from Cloudflare Pages over HTTPS, and communicates with the Render service over HTTPS (REST) and WebSocket Secure (WSS). The Render service connects to MongoDB Atlas over TLS and to the third-party APIs over HTTPS, holding all keys in its environment configuration.

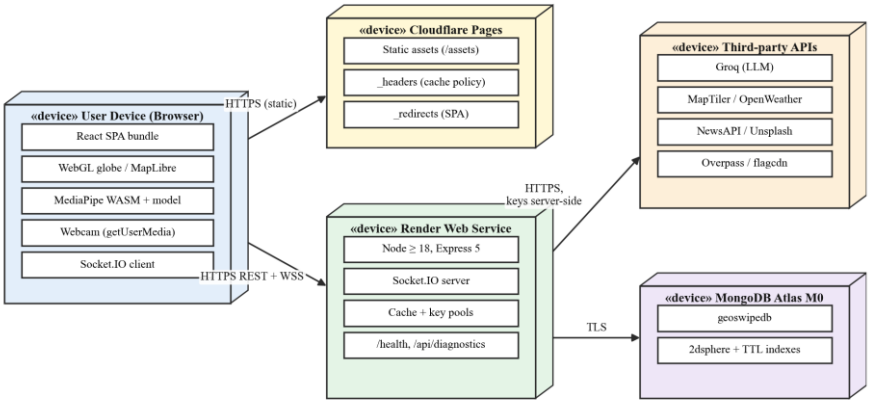


Figure 4.12: Deployment diagram of the production system

4.4 Database Design

MongoDB was chosen because heritage records are naturally nested documents (information, access routes by air, rail and road, media and visitor information) and because it provides geospatial indexing. Table 4.1 lists the collections, and Table 4.2 details the principal HeritageSite document.

Table 4.1: MongoDB collections

Collection	Contents	Records	Indexes
heritagesites	Indian heritage sites with nested details	126	unique name; 2dsphere on coordinates
quiz_questions	Multiple-choice heritage questions (13 monuments × 60)	780	site
countries	Country names for geography games	235	unique name
flag_images	Cached flag URLs and ISO codes	on demand	unique countryKey
monument_images	Cached monument image URLs and source	on demand	unique monumentId
api_cache	Cached weather, news, map-style and safety payloads	on demand	unique (namespace, key); TTL 7 days

Table 4.2: Structure of the HeritageSite document

Field	Type	Description
name	String (unique)	Site name, e.g. “Ajanta Caves”
category	Enum	UNESCO World Heritage, Historic Fort, Rock-cut Cave, Temple, Monument, Palace, Museum, Historic Building
year	String	Period of construction
location	Object	coordinates [longitude, latitude], city, state, country
info	Object	summary, full description, history, architecture, significance, visiting tips
howToReach	Object	summary; byAir (airport, distance); byRail (station, distance); byRoad (routes from major cities, local transport)
media	Object	images, panorama URL, video URL
view360	Object	Embedded 360° street-view URL and summary
model3d	Object	Sketchfab 3D model reference
visitor_info	Object	Timings, entry fee, best time to visit

The quiz bank is balanced by design: each of the 13 monuments has 20 easy, 20 medium and 20 hard questions (780 in total), and each question stores four options and the index of the correct answer. Questions are tagged with a topic; the most frequent topics are architecture (227), history (208), art (55) and conservation (46).

4.5 Interface Design

4.5.1 REST API

Table 4.3 summarises the REST interface of the back end. All routes are prefixed by the server's base URL, which the client reads from a single configuration module.

Table 4.3: REST API endpoints

Endpoint	Purpose
GET /api/heritage-sites, /api/heritage-sites/geojson	All sites; sites as a GeoJSON feature collection for the map
GET /api/heritage-sites/:name/details, /:name/image	Details of one site; resolved image (Unsplash → Wikipedia fallback)
GET /api/heritage/:name, /api/heritage/category/:category	Site by name; sites by category
GET /api/news/:siteName	News with monument → city → state → country fallback
GET /api/country-question, /api/random-flag-country	Geography quiz question; random country with flag
GET /api/quiz/monument/:name, /api/quiz/all-india, /api/quiz/available-monuments, /api/quiz/session/:mode	Heritage quiz questions and sessions
GET /api/safety/nearby-safe-places, /routes, /alerts	Safe places, candidate routes, safety alerts
POST /api/ai/chat; GET /api/ai/status	LLM proxy with profiles *heritage*, *safety*, *tripPlanner*; model status
GET /api/weather/current, /api/weather/forecast	Weather proxy (cached 4 h)
GET /api/maps/style/:styleName, /api/maps/asset/*	MapTiler style (cached 24 h) and tile/sprite/glyph proxy
GET /health, /api/diagnostics, /api/start	Liveness; health of every integration; start-up status

4.5.2 Real-Time Events

Table 4.4 lists the Socket.IO events used by the multiplayer system. The client never decides correctness or scores; it only submits answers, so a modified client cannot award itself points.

Table 4.4: Socket.IO events

Direction	Event	Payload and meaning
Client → Server	join-room	roomId, playerName, gameMode, difficulty; create or join a room
Client → Server	submit-answer	roomId, answer; accepted once per player per round
Client → Server	leave-room, get-room-info	Leave a room; request its state

Direction	Event	Payload and meaning
Server → Client	player-joined, player-left	Updated player list
Server → Client	game-started, new-question	Total rounds, mode, players; question for the round
Server → Client	player-answered	Name and count of players who have answered
Server → Client	show-result	Correct answer, per-player results, scores
Server → Client	game-over, game-ended	Winner or tie with final scores; early termination
Server → Client	room-error, answer-error	Validation or capacity errors

Chapter 5: Methodology

5.1 Introduction

This chapter explains *how* GeoSwipe works. We first describe the iterative development process that we followed, then present the system block diagram, and finally describe each core module with its algorithm (as numbered steps) and the mathematical formulation behind it: in-browser gesture recognition, 3D globe rendering and picking, the two-tier cache, API-key pooling, tiered news retrieval, the route-safety score, multiplayer synchronisation and the AI service profiles. Constants quoted in this chapter are the values used in the deployed source code.

5.2 Development Methodology

GeoSwipe was built iteratively and incrementally rather than in a single waterfall pass, because its modules depend on one another in ways that only become clear during integration: gestures must drive both the globe and every button; the selected monument must be shared by the map, storybook, quiz, chatbot and trip planner; and each external service must be proxied, cached and secured. Figure 5.1 shows the eight phases, each executed as a *develop → integrate → test → refine → extend* cycle. The dates at which each phase was actually delivered are listed in Table 3.9.

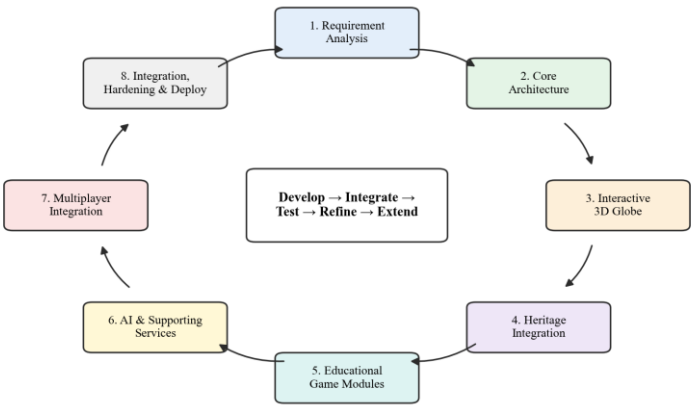


Figure 5.1: Iterative and incremental development methodology followed for GeoSwipe

1. **Requirement analysis** — learning goals were grouped into geographic exploration, heritage information, interactive learning, natural interaction, contextual assistance and multiplayer play.

2. **Core architecture** — the front end, back end, database and third-party services were separated so that modules could be integrated one at a time.
3. **Interactive 3D globe** — the Three.js Earth with shaders, country borders and picking became the primary exploration interface.
4. **Heritage integration** — heritage sites, their coordinates, details and media were stored in MongoDB and rendered on a map with a detail sidebar.
5. **Educational game modules** — country, flag and heritage quizzes were added with difficulty levels, followed by storybooks.
6. **AI and supporting services** — the chatbot, weather, news, trip planner and safety navigation were added around the selected monument.
7. **Multiplayer integration** — Socket.IO rooms turned the quizzes into two-player competitions.
8. **Integration, hardening and deployment** — keys were moved server-side, caching and key pools were added, gesture recognition was moved into the browser and the system was deployed.

5.3 System Block Diagram

Figure 5.2 presents GeoSwipe as input, processing and output blocks. **Inputs** are pointer and keyboard events, webcam hand gestures, natural-language queries and trip preferences, room codes and quiz answers, and, optionally, the device location for safety navigation. **Processing** is divided into six engines. The *gesture engine* converts webcam frames into landmarks, poses and stable events. The *3D globe engine* renders the Earth and resolves which country a click or gesture refers to. The *heritage engine* manages map layers, the monument sidebar and the storybook. The *game engine* generates questions, validates answers and synchronises multiplayer rounds. The *AI engine* builds prompts for the chatbot, the trip planner and the safety guide and parses their responses. The *context engine* supplies weather, news, images and safe places through the cache and key pools and computes route safety scores. **Outputs** are the rendered globe and maps, quiz feedback and scores, monument information with weather and news, narrated story pages, and itinerary PDFs and ranked safe routes. The engines are loosely coupled through two shared channels: a React context that holds the currently selected monument, and the gesture bus that distributes cursor and gesture events.

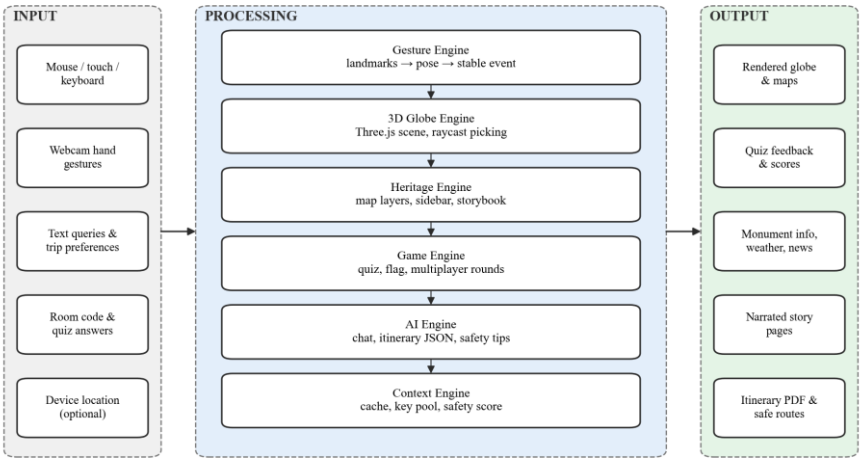


Figure 5.2: System block diagram of GeoSwipe

5.4 In-Browser Hand-Gesture Recognition

5.4.1 Pipeline

Figure 5.3 shows the gesture pipeline. The camera stream (at most 30 FPS, front-facing) is passed to MediaPipe's HandLandmarker [16], [17], which runs in VIDEO mode on the GPU delegate, tracks one hand and uses a minimum detection confidence of 0.7 and a minimum presence confidence of 0.5. For each frame it returns 21 normalised landmarks (x, y, z). The landmarks are mirrored horizontally so that moving the hand to the right moves the cursor to the right, classified into one of nine gestures (or *unknown*) by geometric rules, stabilised, and published on the in-page gesture bus to three consumers: the global cursor, gesture-aware buttons and the globe controller. The WASM runtime and the 7.8 MB model file are self-hosted with the application and downloaded lazily only when a gesture-enabled screen is opened.

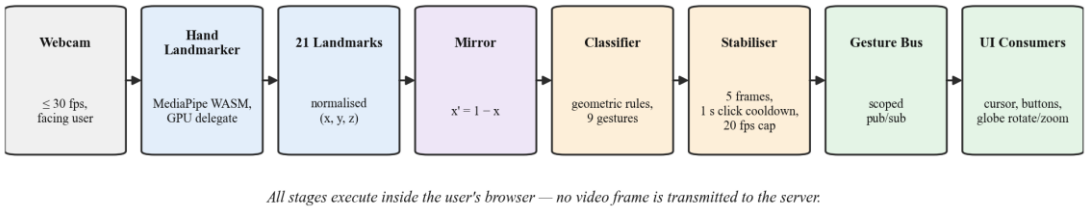


Figure 5.3: In-browser gesture-recognition pipeline

5.4.2 Landmark Geometry

Figure 5.4 identifies the 21 MediaPipe hand landmarks and highlights those used by the classifier. Let $p_i = (x_i, y_i)$ denote landmark i in normalised image coordinates (y increases downwards). The classifier uses the 2D Euclidean distance

$$d(p_i, p_j) = \sqrt{[(x_i - x_j)^2 + (y_i - y_j)^2]} \tag{5.1}$$

and mirrors each landmark before classification:

$$x'_i = 1 - x_i\ ,\quad y'_i = y_i \tag{5.2}$$

A finger f with tip index t and PIP index p is **open** when its tip is above its PIP joint:

$$\text{open}(f) \Leftrightarrow y_t < y_p\ ,\quad f \in \{\text{index } (8, 6), \text{middle } (12, 10), \text{ring } (16, 14), \text{pinky } (20, 18)\} \tag{5.3}$$

The thumb is treated separately because it folds sideways. It is **extended** only when it is far both from its own MCP joint and from the wrist:

$$\text{thumbExt} \Leftrightarrow d(p_4, p_2) > 0.07 \wedge d(p_4, p_0) > 0.12 \tag{5.4}$$

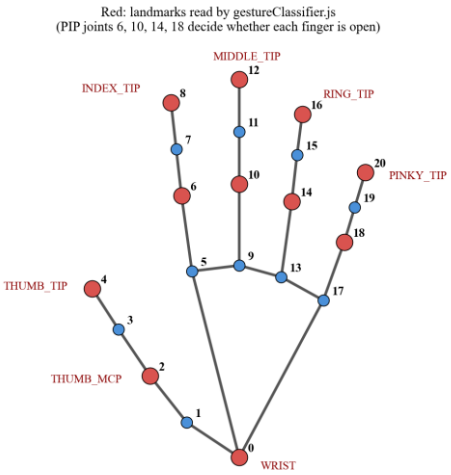


Figure 5.4: MediaPipe hand landmarks; those read by the classifier are highlighted

5.4.3 Classification Rules

Let $\delta = d(p_4, p_8)$ be the thumb–index distance and $\tau = d(p_4, p_0)$ the thumb–wrist distance. The classifier evaluates the rules in Table 5.1 in order and returns the first match; the order resolves ambiguity between similar poses (for example *zoom* must be tested before *index_point*, which is the same hand with the thumb tucked in). Table 5.2 lists the action each gesture triggers in the interface.

Table 5.1: Gesture classification rules (evaluated top to bottom)

Gesture	Condition
click (“OK” sign)	$\delta < 0.05 \wedge$ middle, ring and pinky open

Gesture	Condition
thumbs_up	thumbExt ∧ all four fingers closed ∧ $y_4 < y_0 - 0.08$
thumbs_down	thumbExt ∧ all four fingers closed ∧ $y_4 > y_0 + 0.08$
pinch	$\delta < 0.04$ ∧ middle, ring and pinky closed
zoom (“L” sign)	thumbExt ∧ index open ∧ others closed ∧ $\delta > 0.13$
index_point	index open ∧ others closed ∧ ¬thumbExt ∧ $\tau < 0.10$
cursor_move (open palm)	all four fingers open ∧ $\tau \geq 0.08$
rotate_left / rotate_right	index and middle open, ring and pinky closed; $x_8 < x_0 \rightarrow$ left, $x_8 > x_0 \rightarrow$ right
unknown	none of the above

Table 5.2: Mapping of gestures to interface actions

Gesture	Action
cursor_move	Moves the on-screen cursor to the middle fingertip (landmark 12), which is steadier than a fingertip at the edge of the hand
click	Activates the gesture-aware button under the cursor; selects a country on the globe
rotate_left / rotate_right	Rotates the globe camera about the vertical axis by ∓0.01875 rad per event
thumbs_up / thumbs_down	Rotates the globe camera about the horizontal axis by ±0.0125 rad per event
zoom / pinch	Moves the camera closer (× 0.9875) or farther (× 1.0125), within distance limits of 15 to 100 units

The cursor position is mapped from normalised coordinates to screen pixels with a calibration margin $m = 0.1$, so that the user reaches the screen edges without moving the hand to the edge of the camera frame:

$$X = \text{clamp}((x' - m) / (1 - 2m) \cdot W, 0, W) , \quad Y = \text{clamp}((y - m) / (1 - 2m) \cdot H, 0, H) \tag{5.5}$$

where W and H are the viewport width and height.

5.4.4 Stabilisation Algorithm

Raw per-frame classifications flicker during transitions between poses. The stabiliser (modelled in Figure 4.4) enforces three constraints: a minimum frame interval $\Delta = 50$ ms (at most 20 inferences per second), a stability threshold of $N = 5$ identical consecutive classifications, and a click cool-down $C = 1.0$ s.

Algorithm 5.1 — Gesture stabilisation (executed for every camera frame at time *t*):

1. If $t - t_last < \Delta$, skip the frame and return.
2. Set $t_last \leftarrow t$.
3. If no hand is detected, reset the pose and count, clear the cursor and return.
4. Mirror the landmarks (Eq. 5.2) and classify them (Table 5.1) to obtain gesture *g*.
5. If g equals the previous gesture, increment *count*; otherwise set count ← 1 and remember *g*.
6. If count ≥ N and g ≠ unknown: if g is *click*, emit it only if $t - t_click \geq C$ and then set $t_click \leftarrow t$; otherwise emit g.
7. If g is *cursor_move*, emit the cursor position (landmark 12, Eq. 5.5).

With a camera delivering frames every *T* ms, the frame-skip rule processes one frame every $\lceil \Delta/T \rceil \cdot T$ ms, so the minimum latency from the start of a steady pose to its first emission is

$$L_min = (N - 1) \cdot \lceil \Delta / T \rceil \cdot T \tag{5.6}$$

For a 30 FPS camera ($T \approx 33.3$ ms) this gives $L_min = 4 \times 2 \times 33.3 \approx 267$ ms and an effective inference rate of 15 FPS; Chapter 7 confirms this value by simulation.

5.4.5 Scoped Gesture Bus

Components subscribe to gesture events through getGestureBus(), which returns a new *scope* for each consumer. emit reaches every scope, but off(event) without a handler removes only the calling scope's handlers. This prevents a defect we observed during development, where the globe's clean-up code unsubscribed the global cursor as a side effect when the Explore page mounted. The bus reproduces the isolation that separate Socket.IO connections used to provide, without any network traffic.

5.5 3D Globe Rendering and Picking

The Earth is a sphere of radius 10 units (64 × 64 segments) textured with albedo, bump, ocean and night-light maps, with a cloud layer at radius 10.08 and an atmosphere shell at radius 12.8. The renderer uses ACES filmic tone mapping with an exposure of 1.2. The atmosphere is drawn by a custom GLSL fragment shader: with $\hat{n}$ the surface normal and $\hat{e}$ the view direction in view space,

$$\alpha = (\hat{n} \cdot \hat{e})^{k} \cdot \mu, \quad colour = (0.35 + (\hat{n} \cdot \hat{e})/4.5,\ 0.35 + (\hat{n} \cdot \hat{e})/4.5,\ 1.0) \tag{5.7}$$

where the power factor $k = 4.5$, the multiplier $\mu = 12.0$ and the overall opacity is 0.8, which thickens the blue glow towards the limb of the planet as in the real atmosphere.

For picking, each country polygon from the countrieslite.geo.json file is triangulated with the *earcut* algorithm and projected onto an invisible mesh of radius $R = 10.3$, just above the clouds. A mouse click or gesture click is converted into a ray with THREE.Raycaster; when several meshes are hit, the intersection whose distance from the centre is closest to R is chosen, and the hit point $\mathbf{P} = (x, y, z)$ is converted to geographic coordinates to validate that it lies on the visible hemisphere:

$$\varphi = \sin^{-1}(y / R) \cdot 180/\pi \,, \quad \lambda = \operatorname{atan2}(x, z) \cdot 180/\pi \tag{5.8}$$

The selected country's border then animates through a short colour sequence to confirm the selection. Rendering uses an adaptive frame-rate loop: the actual frame rate f is measured every second, and the target rate is lowered to $\max(30, 0.9 \cdot f)$ when $f < 50$ and restored to 120 when $f > 90$; expensive updates such as lighting run on a subset of frames.

5.6 Two-Tier Cache with Request De-duplication

Weather, news, map styles, safe places and images are requested for a fixed set of places. The cache therefore stores results *per place* and shares them across users. Figure 5.5 shows the getOrFetch(namespace, key, ttl, fetchFn) algorithm.

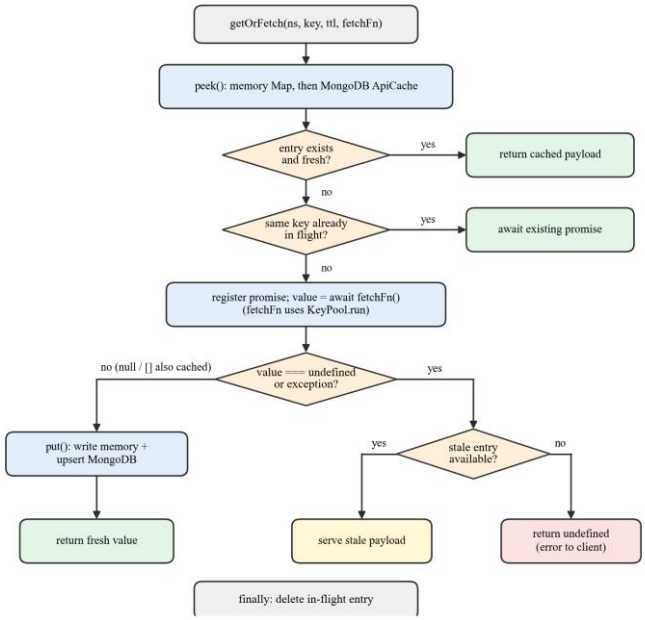


Figure 5.5: Flowchart of the getOrFetch cache algorithm

Algorithm 5.2 — getOrFetch

1. Look up namespace::key in the in-process memory map; if absent and MongoDB is connected, look it up in the api_cache collection and copy it into memory.

2. If an entry exists and now < expiresAt, return it (fresh hit).
3. If a fetch for the same key is already in flight, await and return that promise's result (de-duplication).
4. Otherwise register a new in-flight promise and call fetchFn().
5. If fetchFn returns a value other than undefined, write it to memory and upsert it into MongoDB with expiresAt = now + ttl, then return it. null and empty arrays *are* cached, so a confirmed "no news for this place" is not re-requested on every view.
6. If fetchFn fails or returns undefined, return the expired entry if one exists (stale-on-failure); otherwise return undefined, which the route turns into an error response.
7. Finally, remove the key from the in-flight map.

Expired entries are deliberately kept in MongoDB so that step 6 can serve them; a separate TTL index deletes records not refreshed for seven days to protect the 512 MB database quota. Table 5.3 lists the freshness period chosen for each namespace.

Table 5.3: Cache namespaces and freshness periods

Namespace	TTL	Key and rationale
weather (current, forecast)	4 h	Coordinates rounded to 2 decimals (≈ 1.1 km); 126 sites × 6 refreshes/day stays under 1,000 calls/day
news (per tier)	12 h	Monument, city, state or country tier; shared by all monuments in a place
safety alerts	3 h	Hazard news should be fresher than heritage news
safe places (Overpass)	7 days	Rounded coordinates; hospitals and police stations rarely move
MapTiler style	24 h	Style name; tiles themselves are left to the browser cache
monument / flag images	persistent	Stored in dedicated collections

The benefit can be expressed with a simple model. Without a shared cache, the number of upstream calls per day for a service is $U \times v \times c$, where U is the number of users, v the views per user and c the calls per view. With a per-place cache of freshness period θ hours, it is bounded independently of users by

$$\text{calls/day} \leq S \times \lceil 24 / \theta \rceil \tag{5.9}$$

where S is the number of distinct places requested. For weather, S = 126 and θ = 4 give at most 756 calls per day, below OpenWeatherMap's free limit of 1,000.

5.7 API-Key Pooling

Each service can be configured with several comma-separated keys. Figure 5.6 shows the round-robin algorithm. A pool of *n* keys keeps, for each key, a usage counter and the UTC day on which it was exhausted; counters roll over lazily when the UTC date changes.

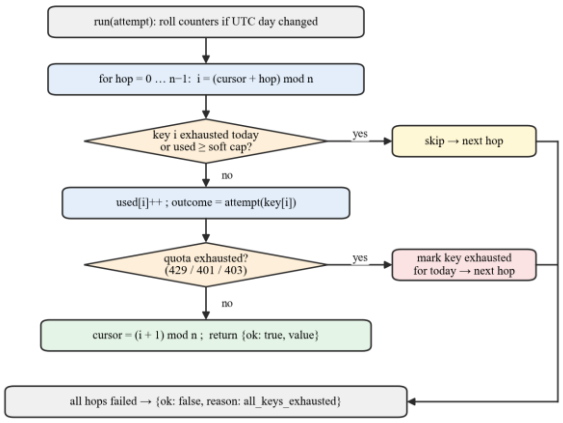


Figure 5.6: Flowchart of the API-key pool

Algorithm 5.3 — KeyPool.run(attempt)

1. If the UTC day has changed, reset all counters.
2. For hop = 0 … n − 1, let i = (cursor + hop) mod n.
3. Skip key *i* if it is marked exhausted today or if its usage has reached the soft cap (90 of NewsAPI's 100 daily requests, 900 of OpenWeatherMap's 1,000, and 3,000 MapTiler requests per day).
4. Increment the usage of key *i* and call attempt(key_i).
5. If the provider signals quota exhaustion (HTTP 429, or 401/403 for a dead key), mark key *i* exhausted for the day and continue with the next hop.
6. Otherwise set cursor ← (i + 1) mod n, so that load is spread evenly, and return the value.
7. If every key has been skipped or exhausted, return all_keys_exhausted, which the cache treats as a failed fetch (and may answer with stale data).

With *n* keys and a soft cap *s* per key, the daily budget for a service is n·s requests, and the cache of Section 5.6 ensures that this budget is consumed by distinct places rather than by repeated views.

5.8 Tiered News Retrieval

Many monuments have little dedicated news coverage. GeoSwipe therefore queries news in up to four tiers, shown in Figure 5.7: the monument itself (L1), its city (L2), its state (L3) and India-wide heritage news (L4), stopping at the first tier that returns articles (up to ten English-language articles, newest first, are requested per query). Each tier is cached separately in the news namespace, so city, state and country tiers are shared by every monument in the same place. The marginal NewsAPI cost of opening the *N*-th monument in a state that has already been visited is therefore typically one call (its own L1 query) or zero.

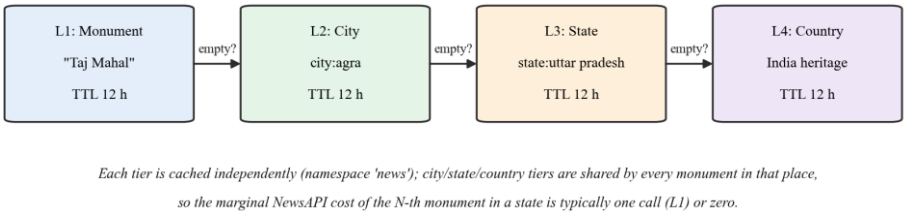


Figure 5.7: Four-tier news retrieval with shared per-tier caching

5.9 Route-Safety Scoring

The safety-navigation module ranks candidate routes by a transparent, rule-based score rather than a learned model, so that users can see why a route is considered safer. Figure 5.8 shows the computation. For a route geometry, up to 12 points are sampled uniformly. For each sample point *q*, the distance to the nearest safe place (hospital, police or fire station returned by the Overpass API within 3 km) is computed with the haversine formula [21]:

$$a = \sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2)\,, \quad d = 2 \cdot r \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{(1-a)}) \qquad (5.10)$$

with r = 6,371,000 m. The coverage score C is one minus the mean nearest distance normalised by 5 km:

$$C = \text{clamp}\left(1 - (1/m) \cdot \Sigma_q \min_s d(q, s) / 5000\,, 0, 1\right) \qquad (5.11)$$

The weather risk W starts at 0.15 and increases by 0.45 for thunderstorms, 0.30 for rain or snow, 0.20 for fog, mist or haze, 0.20 for visibility under 3 km, 0.15 for wind above

10 m/s and 0.15 for temperatures above 41 °C or below 3 °C, clamped to [0, 1]. The alert risk *A* is the maximum severity of active alerts (low 0.2, medium 0.45, high 0.75, critical 1.0). The safety score and the composite ranking score are

$$S = 55 \cdot C + 25 \cdot (1 - W) + 20 \cdot (1 - A) \tag{5.12}$$

$$\text{Composite} = 0.7 \cdot S + 20 \cdot (D_min / D_r) + 10 \cdot (L_min / L_r) \tag{5.13}$$

where D_r and L_r are the duration and distance of route *r* and D_min, L_min are the minima over all candidate routes. Routes are badged *Fastest*, *Shortest* and *Safest*, and the user's chosen preference selects the route to follow. When routing APIs are unavailable, the server generates fallback routes so that the page still functions.

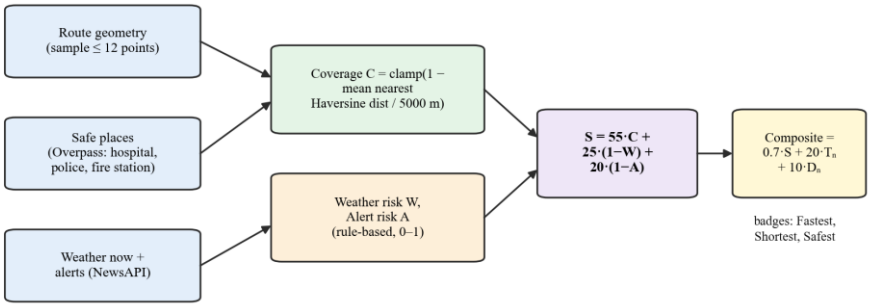


Figure 5.8: Computation of the route-safety and composite scores

5.10 Multiplayer Synchronisation

The server is the single source of truth for every multiplayer game. Rooms are identified by six-character codes drawn from a 32-symbol alphabet that omits look-alike characters (I, O, 0, 1), giving $32^6 \approx 1.07 \times 10^9$ possible codes; the server additionally validates codes against the pattern [A-Za-z0-9_-]{4,64} and sanitises player names to 32 characters.

Algorithm 5.4 — Multiplayer round handling

1. On join-room, validate the code and name, reject the request if the room is full (two players) or if 500 rooms are already active, and add the player.
2. When the second player joins, generate the first question for the room's mode and difficulty and broadcast game-started and new-question.
3. On submit-answer, ignore duplicates from the same socket, evaluate correctness on the server, add one point for a correct answer and broadcast player-answered.

4. When all players have answered, broadcast show-result with the correct answer and scores.
5. If the round number is 10, determine the winner (or a tie), broadcast game-over and reset the room; otherwise, after a 3 s pause, increment the round and go to step 2 for the next question.
6. On disconnect, notify the opponent; delete rooms that are empty or inactive for more than 10 minutes (checked every minute).

Room actions are limited to 30 events per 10 s per socket by a token-bucket limiter, and the Socket.IO message size limit is 64 KB, so a single client cannot exhaust server memory.

5.11 AI Services

All LLM requests go through one server route, POST /api/ai/chat, which accepts a *profile* name and the conversation, never the model or sampling parameters; these are fixed on the server so that a modified client cannot change them. Table 5.4 lists the profiles. The model identifier is read from the server configuration (currently openai/gpt-oss-120b on Groq), and a separate /api/ai/status route verifies that the configured model is available on the account. For the heritage profile, the server appends the names and categories of up to 50 heritage sites from the database to the system prompt so that answers are grounded in the platform's own catalogue. The trip-planner profile requests a JSON object that follows a fixed schema — title, summary, weather advice, day-wise schedule with timings, food and travel suggestions, a cost breakdown in Indian rupees and booking links — which the client parses, normalises and renders, and which can be exported to PDF with jsPDF.

Table 5.4: LLM profiles enforced by the server

Profile	Temperature	Max tokens	Purpose and output
heritage	0.7 (top-p 0.9)	2,000	Conversational answers about Indian heritage and travel
safety	0.4	1,500	Cautious route-safety guidance
tripPlanner	0.6 (JSON mode)	4,000	Structured day-wise itinerary with INR cost breakdown

Token budgets are deliberately generous because the configured model is a reasoning model whose internal reasoning tokens count against the limit; the server detects an

empty completion and reports a specific hint rather than a generic error. AI requests are additionally limited to 20 per 5 minutes per IP address.

Chapter 6: Implementation Details

6.1 Introduction

This chapter documents the implemented system. Section 6.2 walks through every user-facing module with screenshots captured from the production deployment at https://geoswipe.pages.dev on 30 September 2026 (1440 × 900 viewport, Google Chrome). Sections 6.3 to 6.6 describe the front-end and back-end code organisation, the security measures, and the storybook content pipeline. Sections 6.7 and 6.8 cover deployment and testing.

6.2 System Implementation

6.2.1 Landing Page and Navigation

The landing page (Figure 6.1a) introduces GeoSwipe with two entry cards — **Interactive Globe** (explore Earth in 3D with games, quizzes and multiplayer challenges) and **Heritage Mode** (explore cultural heritage sites) — and a navigation bar with Features, Gesture Controls, About and Contact links. The small floating toolbar at the top right, present on every page, toggles the menu, the light/dark theme and the welcome guide; the camera button at the bottom left starts or hides the webcam for gesture control. Scrolling down reveals the **Hand Gesture Controls** section (Figure 6.1b), which illustrates each supported gesture — thumbs up/down to rotate vertically, two fingers right/left to rotate horizontally, pinch to zoom out, the "L" sign to zoom in, the open palm to move the cursor and the "OK" sign to click — followed by an *About GeoSwipe* section.

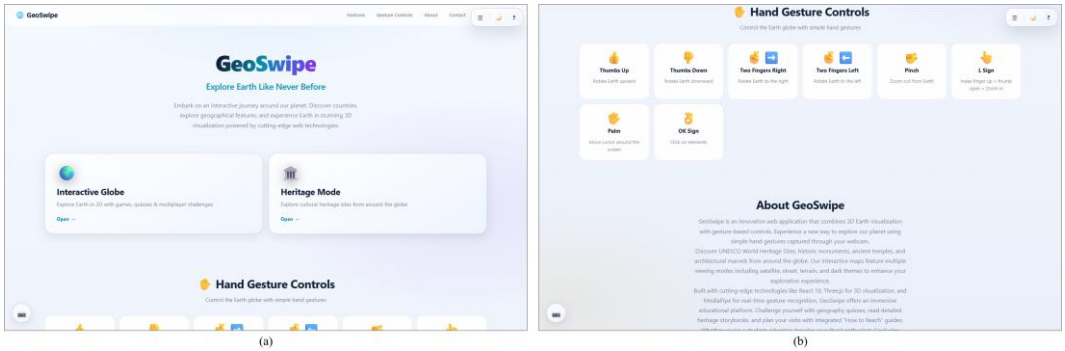
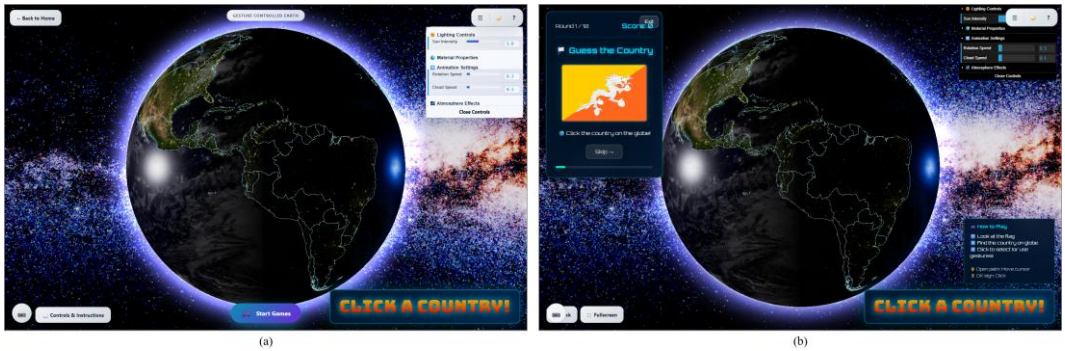


Figure 6.1: (a) Landing page with mode cards and global toolbar; (b) hand-gesture guide on the landing page

6.2.2 Explore Mode and the Flag Game

Figure 6.2a shows Explore mode. The Three.js globe is rendered with day and night textures, a cloud layer, a starfield background and the shader-based atmospheric glow described in Section 5.5; country borders are drawn on the surface. The *Gesture Controlled Earth* label indicates that the globe accepts gesture input, and the dat.GUI panel at the top right exposes lighting intensity, material properties, rotation and cloud speed, and atmosphere effects; **Start Games** opens the game-selection modal. In the **flag-guessing game** (Figure 6.2b) the learner sees a flag — here, Bhutan — with the round counter (round 1 of 10) and score, and must *click the country on the globe*. Answering on the globe rather than choosing from a list forces the learner to locate the country, tying the game mechanic to geographic knowledge as recommended in [2]. A *How to Play* panel lists both mouse and gesture controls.

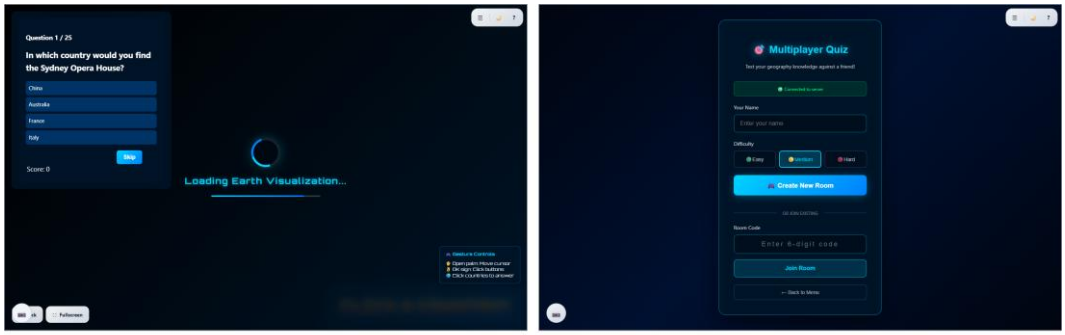


(a) (b)

Figure 6.2: (a) Explore mode with the interactive 3D globe and rendering controls; (b) flag-guessing game answered by clicking the country on the globe

6.2.3 Country Quiz and Multiplayer Lobby

The **country quiz** (Figure 6.3a) asks multiple-choice questions such as *"In which country would you find the Sydney Opera House?"*, with a skip option and running score, after the learner selects a difficulty level. Figure 6.3b shows the multiplayer quiz lobby. The status badge confirms that the Socket.IO connection is established. A player enters a name, chooses a difficulty and either creates a new room — which generates a six-character code to share — or joins an existing room by entering its code. The multiplayer flag game and heritage quiz use lobbies of the same design; once two players are present, the game proceeds through the ten synchronised rounds described in Section 5.10.

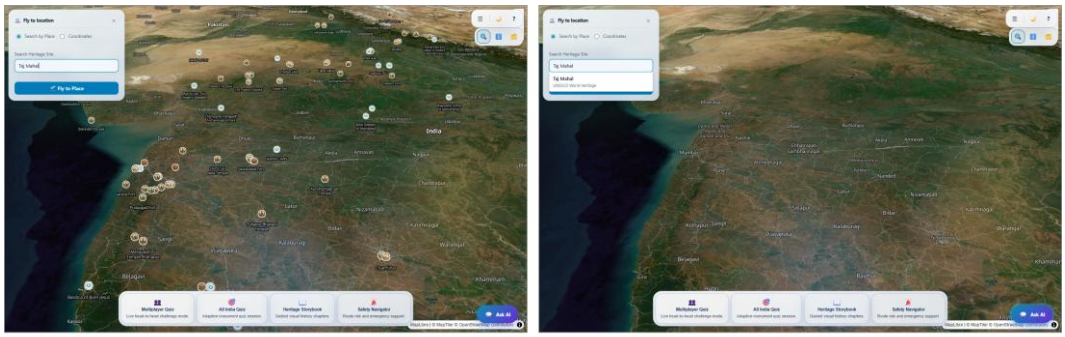


(a) (b)

Figure 6.3: (a) Country quiz with multiple-choice options; (b) multiplayer lobby for creating or joining a room

6.2.4 Heritage Map

Heritage mode (Figure 6.4a) displays all 126 heritage sites on a MapLibre GL map with satellite imagery and a distinct icon for each category. The action cards at the bottom open the multiplayer quiz, the all-India quiz, the storybook library and the safety navigator, and the **Ask AI** button opens the heritage assistant. The **Fly to location** panel (Figure 6.4b) lets the learner search for a site by name, with auto-complete suggestions showing the site's category, or enter coordinates, and then flies the camera to the site. Satellite, hybrid, topographic, street, dark and historical styles are available; all style documents are fetched through the server's MapTiler proxy.



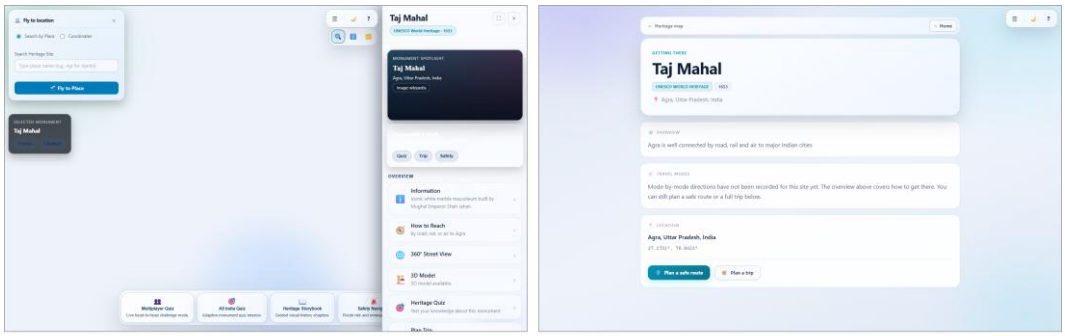
(a) (b)

Figure 6.4: (a) Heritage map of India with category icons for all sites; (b) searching a site with auto-complete in the Fly-to panel

6.2.5 Monument Sidebar and How to Reach

Selecting a site opens the monument sidebar (Figure 6.5a). Its header shows the name, category and year (Taj Mahal, UNESCO World Heritage, 1653) and a *Monument spotlight* card with the location and an image resolved by the server from Unsplash or, as a fallback, Wikipedia. Quick buttons start the monument's quiz, a trip plan or safety navigation. The *Overview* list gives access to detailed information (history, architecture, significance), *How to Reach*, the 360° street view, the 3D model where available, the heritage quiz, trip planning and the weather (current and 5-day forecast)

and news panels. The *Selected Monument* chip keeps the selection visible and offers *Focus* and *Chatbot* shortcuts. The *How to Reach* page (Figure 6.5b) presents a travel overview, travel modes by air, rail and road when recorded, and the location with coordinates (27.1751° N, 78.0421° E for the Taj Mahal), with buttons to plan a safe route or a full trip.

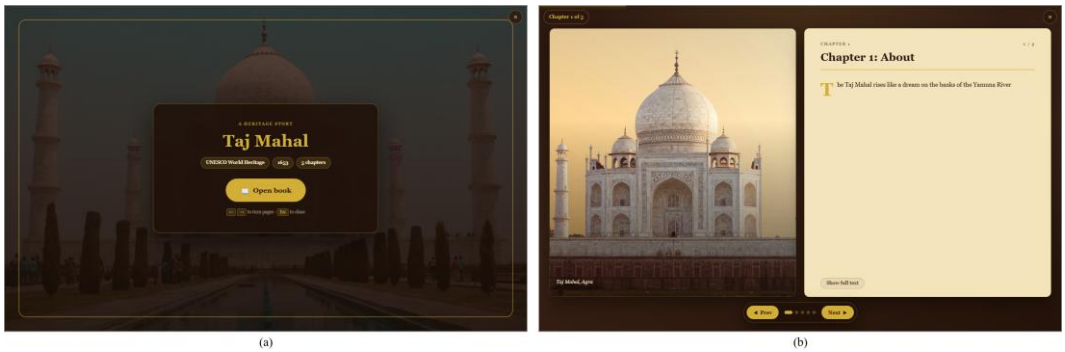


(a) (b)

Figure 6.5: (a) Monument sidebar for the Taj Mahal; (b) How-to-Reach page for the selected monument

6.2.6 Heritage Storybook

Storybooks present a monument's story as an illustrated, narrated book. The cover (Figure 6.6a) shows the title, category, year and number of chapters over a full-bleed photograph. Opening the book (Figure 6.6b) reveals a two-page spread built with react-pageflip: the left page shows the chapter illustration with a caption, and the right page shows the chapter title and text, which appears with a typewriter effect synchronised to the chapter's narration audio. *Prev* and *Next* buttons, the arrow keys and gesture clicks turn pages, with a page-turn sound effect.



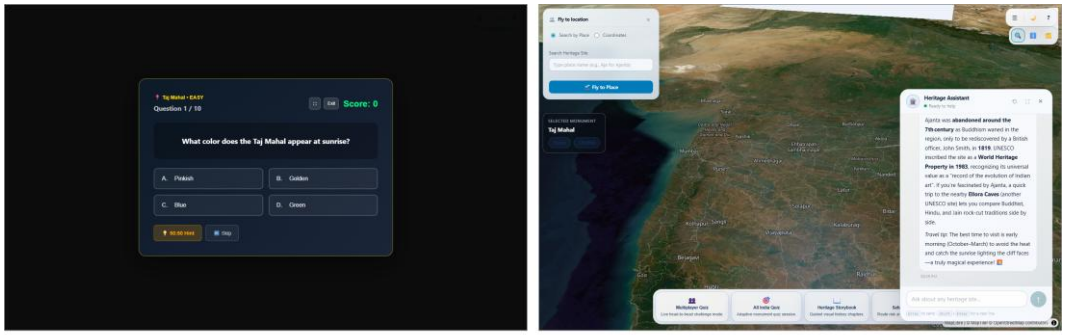
(a) (b)

Figure 6.6: (a) Storybook cover for the Taj Mahal; (b) open spread with illustration and narrated chapter text

6.2.7 Heritage Quiz and AI Heritage Assistant

The heritage quiz (Figure 6.7a) can be taken for a specific monument or for all of India. After choosing a difficulty, the learner answers ten questions drawn from the 780-question bank, for example *"What color does the Taj Mahal appear at sunrise?"*. The

header shows the monument, difficulty, question number and score; a *50:50* hint that removes two wrong options (once per question) and a skip button are available. Figure 6.7b shows the heritage assistant answering a live query, *"Tell me briefly about the history of the Ajanta Caves."* The response, generated by the LLM through the server's heritage profile, is rendered with rich text and includes a practical travel tip; the widget keeps the conversation history and can be expanded.



(a) (b)

Figure 6.7: (a) Monument-specific heritage quiz question; (b) AI heritage assistant answering a question about the Ajanta Caves

6.2.8 AI Trip Planner and Safety Navigation

The trip planner (Figure 6.8a) combines the selected site with the learner's inputs: number of days and travellers, start date, budget level (low, medium, high or custom), origin city and preferred mode of travel. *Generate Itinerary* sends the site context, the inputs and a weather summary to the tripPlanner profile; the result panel shows a day-by-day schedule with timings, a cost estimate for stay, food, transport and entry fees in rupees, and booking links, and the plan can be exported as a PDF. The *Smart Tourist Safety and Emergency Navigation* page (Figure 6.8b) lets the learner search for a destination, choose a route strategy (safest, fastest or shortest) and generate candidate routes ranked with the safety score of Section 5.9. Emergency and evacuation modes and live navigation are provided on the left; the right-hand panel lists nearby safe places, weather and alerts, and emergency contacts for India — national emergency 112, police 100, ambulance 108, fire 101 and women's helpline 1091 — with custom contacts. An AI guidance box accepts questions such as *"Is this route safe at night?"*.

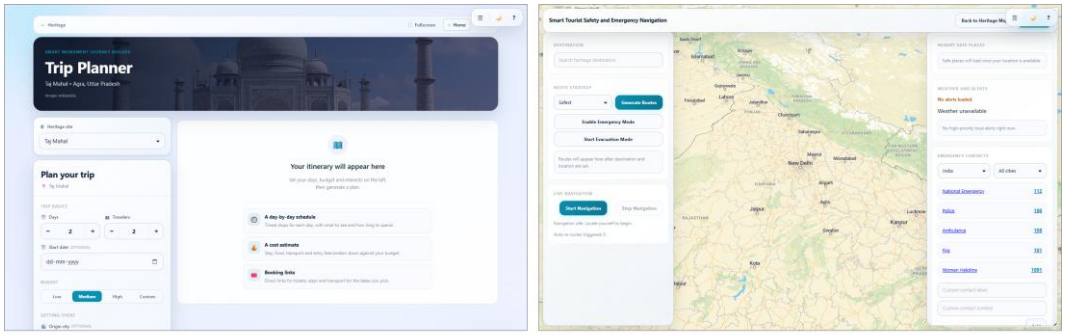


Figure 6.8: (a) AI trip-planner input form; (b) safety-navigation page with route strategy, map and emergency contacts

6.3 Front-End Implementation

The client is organised as page components in client/src, reusable components in components/, shared state in context/ (theme and heritage selection), hooks in hooks/ (full-screen control), service modules in utils/ and styles in styles/. Table 6.1 lists the largest modules and their responsibilities. In total the client contains 21,040 lines of JavaScript/JSX and 7,480 lines of CSS.

Table 6.1: Principal front-end modules

Module	LOC	Responsibility
HeritagePage.jsx	3,694	Heritage map, layers, icons, styles, sidebar panels, weather/news, fly-to
EarthThreeJS.jsx	1,885	Three.js scene, shaders, country meshes, raycast picking, gesture mapping, adaptive FPS
HeritageQuiz.jsx / HeritageMultiplayerQuiz.jsx	1,278 / 1,144	Single-player and multiplayer heritage quizzes
SafetyNavigationPage.jsx	1,134	Route generation, ranking, safe places, contacts, AI guidance
HeritageStoryBook.jsx	680	Page-flip reader, narration audio, typewriter text
FlagGuessGame.jsx / MultiplayerFlagGame.jsx	534 / 524	Flag game on the globe; two-player version
utils/tripPlannerService.js, tripPlanPdf.js	486 / 210	Trip-plan orchestration; PDF export
components/HeritageChatbot.jsx	452	Chat widget and conversation history
components/CameraCapture.jsx	307	Webcam access and MediaPipe HandLandmarker loop
utils/safetyScoring.js	253	Haversine, coverage, risk and composite scores
utils/multiplayerSocket.js	217	Shared Socket.IO connection and room helpers
utils/gestureClassifier.js / gestureBus.js	209 / 108	Gesture rules and stabiliser; scoped event bus

Performance-related choices in the front end include route-level code splitting with React.lazy, a dynamic import of MediaPipe so that its 140 KB bundle (42 KB compressed) loads only when a gesture screen opens, self-hosting of the WASM runtime and model, reduced sphere tessellation for the Earth (64 × 64) and clouds (32 × 32), and the adaptive rendering loop of Section 5.5.

6.4 Back-End Implementation

The server entry point server/index.js configures Express, Mongoose and Socket.IO and implements the heritage, quiz, flag, news, safety and multiplayer logic. Cross-cutting concerns are separated into modules, listed in Table 6.2. The server contains 4,734 lines of JavaScript excluding data and seed files.

Table 6.2: Back-end modules

Module	LOC	Responsibility
index.js	2,129	App set-up, REST routes, news tiers, safety routes, multiplayer rooms
routes/aiProxy.js	355	LLM profiles, model status, key redaction, AI rate limit
routes/mapProxy.js	326	MapTiler style/asset proxy, URL rewriting, path validation
routes/diagnostics.js	262	Health probes for all integrations with fix hints
routes/weatherProxy.js	168	Weather proxy with 4 h cache and key pool
services/cache.js	177	Two-tier cache, in-flight de-duplication, stale-on-failure
services/keyPool.js	125	Round-robin key pool with soft caps and daily roll-over
services/monumentImageService.js + providers	500	Unsplash → Wikipedia image resolution and caching
config/env.js	180	Environment parsing, key lists, CORS wildcard matching
middleware/security.js, validation.js	54 / 48	Security headers, safe errors; regex escaping, sanitisation

The heritage and quiz data are loaded into MongoDB by seed scripts (mongo_data_for_heritage_sites.js with 126 site documents and heritage_quiz_data.js with 780 questions). Country names and ISO 3166-1 alpha-2 codes for the 235 countries used by the geography games are stored as a static module generated from the same GeoJSON file that draws the globe, which removed an earlier dependency on a third-party country API that had been deprecated.

6.5 Security Implementation

Security was treated as a design requirement after an internal audit of an earlier version found API keys compiled into the browser bundle through VITE_* variables. The implemented measures are:

- **Server-side secrets.** The only variable exposed to the browser is the API base URL. Groq, OpenWeatherMap, MapTiler, NewsAPI and Unsplash are reached only through server proxies, and upstream error details are passed through a redaction function that masks key-shaped strings before logging.
- **Security headers.** Every API response sets X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: no-referrer, a restrictive Permissions-Policy, Content-Security-Policy: default-src 'none'; frame-ancestors 'none'; sandbox and Cross-Origin-Resource-Policy: same-site; HSTS is added in production and the X-Powered-By header is removed. Map responses relax the CSP only as far as MapLibre requires.
- **CORS allow-list** for both HTTP and WebSocket traffic, with a single-label wildcard for Cloudflare Pages preview URLs, matched on a parsed URL. It was verified against 15 cases, including look-alike hosts such as geoswipe.pages.dev.evil.com, protocol downgrades and port mismatches, all of which are rejected.
- **Input validation.** User input used in MongoDB regular expressions is escaped to prevent NoSQL regex injection and regular-expression denial of service; free-text fields are trimmed, length-capped and stripped of control characters.
- **Rate limiting** per route group and per socket, and a cap of 500 active rooms (Chapter 3, Table 3.8).
- **Safe errors.** Clients receive generic error messages; internal details are logged only on the server.
- **Webcam privacy.** Gesture inference runs in the browser and the front end declares Permissions-Policy: camera=(self), geolocation=(self).

6.6 Storybook Content Pipeline

Storybooks are data-driven. Each monument has a JSON file in client/public/chapters/ containing an ordered list of chapters, each with a title, narrative text and image

metadata (URL, caption, focal point and fit). Hand-authored books currently exist for six monuments — Ajanta Caves, Gateway of India, Raigad Fort, Shaniwar Wada, Taj Mahal and Trimbakeshwar Temple — with five chapters each, i.e. 30 illustrated chapters with 30 narration MP3 files. A manifest script regenerates the index of available books. The Python script audio-to-captions.py uses the OpenAI Whisper *base* model with word-level timestamps to generate captions from the narration audio, which supports synchronising text with speech.

6.7 Deployment

The production system is deployed on free infrastructure as shown in Figure 4.12:

1. **Database.** A MongoDB Atlas M0 cluster is created and the heritage and quiz seed scripts are run against it.
2. **Back end.** The server/ folder is deployed to Render as a Node web service in the Singapore region using the render.yaml blueprint, with HOST=0.0.0.0, NODE_ENV=production and a /health check path. Secrets (MongoDB URI, pooled API keys, Groq key and model, allowed origins) are entered in the Render dashboard and never committed to Git.
3. **Front end.** The client/ folder is built by Cloudflare Pages with VITE_API_URL set to the Render URL. A pre-build step copies the MediaPipe WASM runtime into public/mediapipe/ and downloads the hand-landmark model so that both are served from the application's own origin. _redirects provides the SPA fallback and _headers sets immutable caching for hashed assets and 30-day caching for MediaPipe files.
4. **Verification.** /api/diagnostics is queried to confirm that MongoDB and every integration respond, and the deployment checklist (globe, map, weather, news, AI, multiplayer and gestures) is exercised in a browser.

6.8 Testing

Testing combined static checks, differential and behavioural tests of the gesture module, API checks and end-to-end checks in a browser. Table 6.3 lists the test cases and their outcomes. Test cases T1–T7 were performed by the team during the September 2026 hardening; T8–T13 were performed for this report on 30 September 2026 against the production deployment.

Table 6.3: Test cases and results

ID	Test	Method / expected result	Result
T1	Lint	eslint src: no errors	Pass (0 errors)
T2	Production build	vite build succeeds; MediaPipe split into a lazy chunk	Pass
T3	Gesture port fidelity	Python vs JavaScript classifier on 20,000 generated poses: identical labels	Pass (0 mismatches)
T4	Stabiliser behaviour	15 checks: 5-frame threshold, 1 s cooldown, 50 ms cap, cursor clearing, bus semantics	Pass
T5	Gesture-bus scoping	15 checks incl. Explore mount/unmount; other consumers unaffected by off(event)	Pass
T6	Cache correctness	Each cached endpoint called twice: identical payloads, warm call faster	Pass
T7	CORS wildcard	15 origin cases incl. look-alike domains and protocol downgrade	Pass (all attacks denied)
T8	Live API endpoints	12 endpoints × 10 requests from client: HTTP 200	Pass (120/120)
T9	Integration health	/api/diagnostics on production	9/10 OK; Wikipedia fallback throttled (HTTP 429, optional)
T10	Page rendering	13 routes loaded in headless Chrome; screenshot captured	Pass (13/13)
T11	AI assistant	Live question answered through /api/ai/chat	Pass
T12	Gesture templates	Each of 9 canonical poses classified as its own label with zero noise	Pass (9/9)
T13	Gesture on real camera	Recognition with a physical webcam and real hands	[INSERT: result of manual test with real webcam]

Test T13 could not be automated because the sandboxed browser used for automated testing does not grant camera access; the classifier, stabiliser and bus were therefore verified with the differential and synthetic tests (T3–T5, T12) and the gesture robustness analysis of Chapter 7, and a manual test on a physical webcam must be recorded by the team.

Chapter 7: Result Analysis

7.1 Introduction

This chapter evaluates GeoSwipe quantitatively. Because the platform integrates many services, we evaluate it along the dimensions that determine whether it works for real users: (i) server-side cache effectiveness and third-party API consumption, (ii) end-to-end latency and health of the deployed API, (iii) robustness, latency and throughput of gesture recognition, (iv) network cost and privacy of the gesture design, and (v) front-end bundle size and page load. We then compare GeoSwipe with the systems reviewed in Chapter 2.

Two sources of data are used, and each table states which applies. **Team measurements** were recorded by the project team on 12 September 2026 while hardening the server, against the server running with MongoDB and pooled keys. **Report measurements** were taken for this report on 30 September 2026 against the production deployment (front end on Cloudflare Pages, back end on Render in Singapore) from the development laptop described in Table 3.2, over a residential broadband connection in Mumbai **[ASSUMED: network type and location of the measuring client]**. Gesture-robustness results come from a synthetic benchmark that exercises the project's unmodified gestureClassifier.js in Node.js 22.16; it does **not** involve human participants or real camera images, and we discuss this limitation in Section 7.5. No usability or learning-gain study has yet been conducted.

7.2 Performance Measures

7.2.1 Server-Side Cache Effectiveness

Table 7.1 and Figure 7.1 compare the server response time of each cached endpoint on a cold cache (the server must call the upstream provider) and a warm cache (the result is served from memory). Warm responses took 2–6 ms, 20× to 4,363× faster than cold responses. The largest gain is for nearby safe places, whose Overpass query took 8.7 s cold; because every user shares the server's IP address, caching also prevents the public Overpass instance from throttling the whole application. The payloads of the cold and warm calls were identical apart from the fetchedAt timestamp.

Table 7.1: Cold versus warm server response time of cached endpoints (team measurement)

Endpoint	Cold (ms)	Warm (ms)	Speed-up
weather/current	235	6	39×
weather/forecast	78	4	20×
maps/style	672	5	134×
safety/nearby-safe-places	8,726	2	4,363×
safety/alerts	347	4	87×

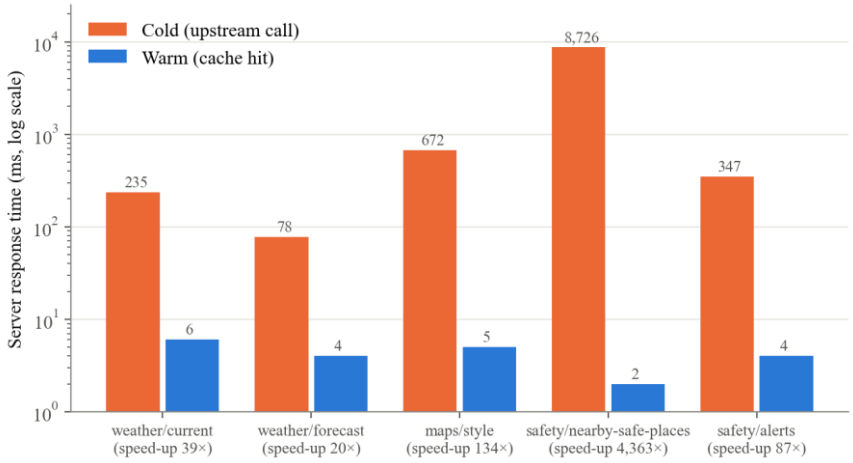


Figure 7.1: Cold versus warm response times of cached endpoints (logarithmic scale)

7.2.2 Third-Party API Consumption

NewsAPI's free tier allows 100 requests per day, and before the tiered cache a single monument view could trigger up to eight requests. Table 7.2 reproduces the team's measurement of three monuments in Maharashtra followed by a repeat of the first. The whole sequence consumed **5 NewsAPI calls**, compared with an estimated 32 calls for the same sequence under the earlier design; the third site's location panel reused the city:mumbai tier populated by the first site, and the repeat view cost nothing and returned in 18 ms.

Table 7.2: NewsAPI usage for a four-view sequence with tiered caching (team measurement)

Monument viewed	Time (ms)	Monument articles (tier)	Location articles
Kanheri Caves	706	5 (L2)	5
Mahalakshmi Temple, Kolhapur	656	1 (L1)	5
Chhatrapati Shivaji Terminus	355	5 (L1)	5
Kanheri Caves (repeat)	18	5 (cached)	—

For weather, Eq. (5.9) bounds daily consumption at 126 sites × ⌈24/4⌉ = 756 calls, which is below the 1,000-call free limit regardless of the number of users; with n pooled keys the headroom grows to $900 \cdot n$ calls per day. The same reasoning applies to map styles (24 h) and safe places (7 days).

7.2.3 End-to-End Latency of the Deployed API

Table 7.3 reports the end-to-end response time seen by a client for 12 production endpoints, each requested 10 times at 0.3 s intervals after the service had been woken. The *first* column is the first request to that endpoint in the run; median, mean and 95th percentile are computed over the remaining nine requests. All 120 requests returned HTTP 200. The Render free instance had been idle, so the very first /health request took **23.1 s** (a cold start of the service); subsequent /health requests had a median of 383 ms, which approximates the network round-trip between the client and the Singapore region plus TLS overhead.

Table 7.3: End-to-end latency of production API endpoints (report measurement, n = 10 per endpoint)

Endpoint	First (ms)	Median (ms)	Mean (ms)	P95 (ms)	Size (bytes)
/health	831	383	467	907	41
/api/weather/current	889	351	347	391	497
/api/maps/style/:styleName	1,639	371	437	819	62,715
/api/random-flag-country	1,019	386	437	722	71
/api/weather/forecast	1,049	480	794	2,097	16,345
/api/country-question	948	539	558	973	145
/api/heritage-sites/:name/details	704	566	573	674	3,953
/api/quiz/available-monuments	646	573	603	774	196
/api/news/:siteName	1,808	648	743	1,186	6,158
/api/quiz/all-india	800	802	874	1,416	180
/api/heritage-sites/geojson	1,683	1,401	1,433	1,604	29,937
/api/heritage-sites	2,113	2,233	2,235	2,921	271,926

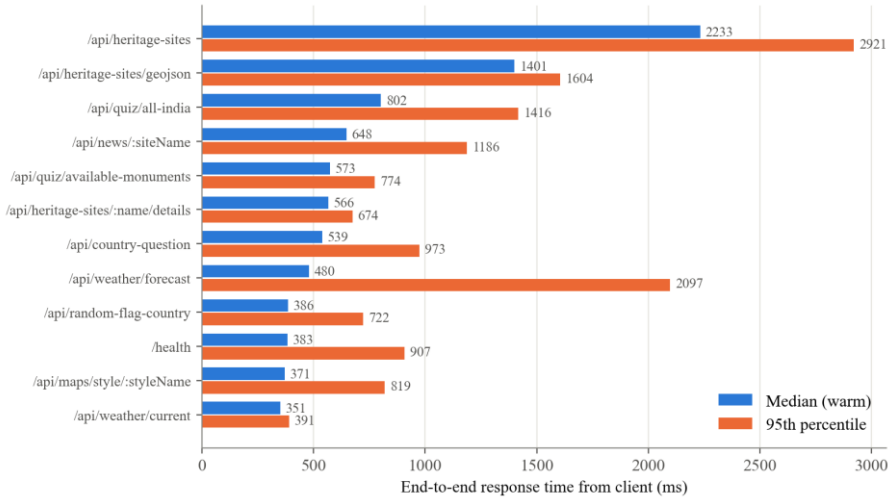


Figure 7.2: Median and 95th-percentile end-to-end latency of production endpoints

Three observations follow from Figure 7.2. First, cached endpoints (weather, map style, news) have warm medians of 351–648 ms, only slightly above the /health baseline, confirming that once the cache is warm the network dominates. Their *first* requests were 2–5× slower, consistent with cache misses. Second, the two slowest endpoints return the full site catalogue: /api/heritage-sites transfers 272 KB of nested documents and /api/heritage-sites/geojson 30 KB; the map uses the lighter GeoJSON endpoint, and both are candidates for response caching or projection of fewer fields. Third, the occasional P95 spikes (e.g. 2.1 s for the forecast) are typical of a shared free instance and residential network and do not recur in the median.

7.2.4 Integration Health

The health probes returned by /api/diagnostics on the production system took between 413 ms (flagcdn) and 948 ms (MongoDB) per integration, except the Overpass probe, which took 4,873 ms. Nine of ten integrations were healthy, including every required one (MongoDB, Groq, OpenWeatherMap, NewsAPI, MapTiler, the trivia API and flagcdn). The only failing probe was the optional Wikipedia image fallback, which returned HTTP 429 (rate limited) at the time of measurement; monument images continued to be served from the Unsplash provider and the image cache. The Groq probe confirmed that the configured model was available among 11 models on the account.

7.2.5 Gesture-Recognition Robustness

Benchmark design. We constructed one canonical 21-landmark template for each of the nine gestures in normalised image coordinates and verified that each template is

classified as its own label (test T12). For each gesture and noise level σ we generated 2,000 perturbed poses by adding independent Gaussian noise $N(0, \sigma^2)$ to the x and y coordinates of every landmark, and classified them with the project's unmodified classifier — 18,000 poses per noise level. σ is expressed in normalised image units; σ = 0.01 corresponds to about 6 pixels on a 640-pixel-wide frame. A fixed random seed makes the benchmark reproducible.

Table 7.4: Classification accuracy versus landmark noise (synthetic benchmark, 2,000 poses per gesture per level)

Noise σ	Overall accuracy (%)	Worst gesture (%)	Worst gesture
0	100.00	100.00	—
0.0025	100.00	100.00	—
0.005	100.00	100.00	—
0.0075	99.87	98.90	pinch
0.010	99.06	94.95	pinch
0.015	91.03	72.35	pinch
0.020	78.46	45.00	pinch
0.030	57.75	18.05	pinch

Table 7.4 shows that per-frame accuracy stays at 100 % up to σ = 0.005 and at or above 99.06 % up to σ = 0.01, then degrades as noise approaches the size of the classifier's distance thresholds (0.04–0.13). The per-gesture heat map in Figure 7.3 shows that *pinch* is the most noise-sensitive gesture, because its thumb–index threshold (0.04) is the tightest in Table 5.1, while *cursor_move* (open palm) and *thumbs_down* are the most robust. At σ = 0.01 the dominant confusions were *pinch* → *unknown* (96 of 2,000), *click* → *cursor_move* (15) and *thumbs_up* → *unknown* (11); errors that fall into *unknown* are benign because *unknown* never triggers an action.

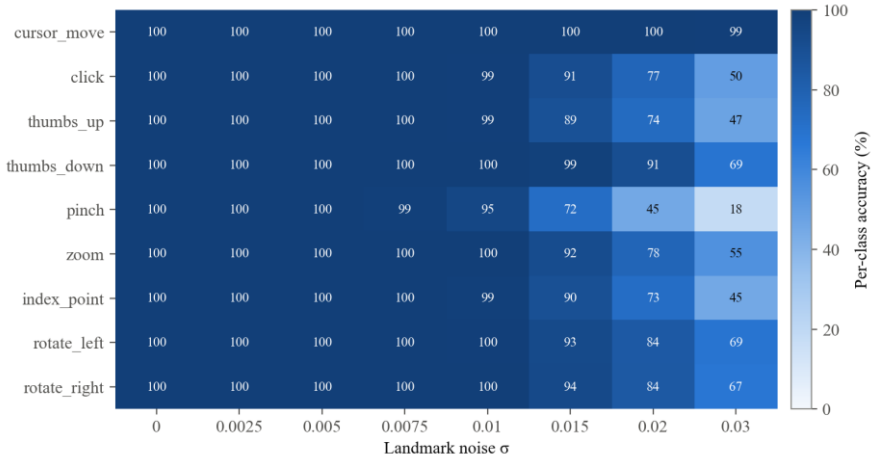


Figure 7.3: Per-gesture accuracy (%) at each noise level

Because the classifier's thresholds are absolute distances in image coordinates, the apparent size of the hand — which depends on the distance from the camera — could affect accuracy. We therefore scaled each template about the wrist by factors from 0.5 to 1.6 at σ = 0.005. Overall accuracy stays between 99.38 % and 100 % over this range; the worst gesture falls to 98.35 % (zoom) at half size and 97.60 % (index_point) at 1.6× size, so users should keep the hand at a moderate distance from the camera.

Effect of the stabiliser. To measure the stabiliser, we simulated a user holding a gesture for 3 s in front of a 30 FPS camera (300 runs per condition), with fresh noise on every frame, and fed the frames through the project's stabiliser. Table 7.5 and Figure 7.4 compare the raw per-frame misclassification rate with the fraction of *emitted* gestures that were wrong. Even when 53.8 % of raw frames were misclassified (pinch at σ = 0.02), **no wrong gesture was emitted in any condition**: requiring five identical consecutive classifications converts classification noise into delay rather than into wrong actions. The cost is latency: the median time to the first emission rises from 267 ms to 1.4 s at the highest noise, and in 185 of 300 runs at that level no gesture was emitted within 3 s.

Table 7.5: Stabiliser effect under noise (simulated 30 FPS camera, 300 runs of 3 s per condition)

Gesture @ σ	Raw frame error (%)	Wrong emitted (%)	Median latency (ms)	P95 latency (ms)
pinch @ 0.010	5.56	0	267	600
pinch @ 0.015	28.20	0	667	2,133
pinch @ 0.020	53.81	0	1,400	2,667
thumbs_up @ 0.010	0.92	0	267	267
thumbs_up @ 0.015	11.63	0	267	867

Gesture @ σ	Raw frame error (%)	Wrong emitted (%)	Median latency (ms)	P95 latency (ms)
thumbs_up @ 0.020	28.45	0	733	2,333

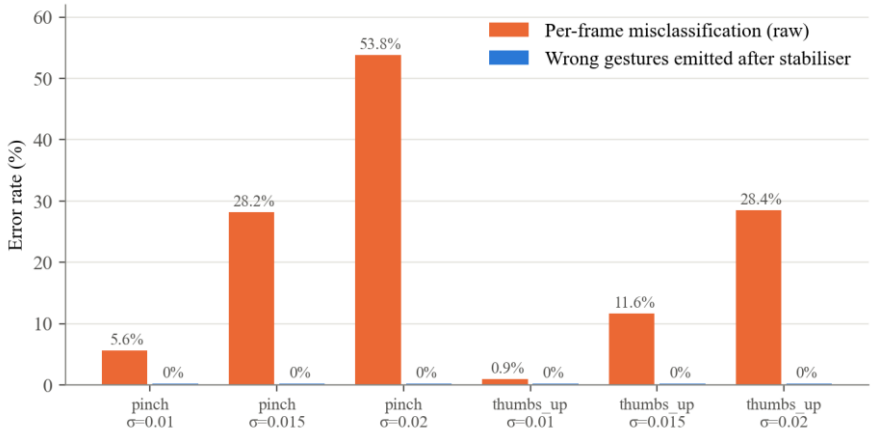


Figure 7.4: Raw per-frame error versus wrong gestures emitted after stabilisation

Table 7.6 summarises the timing behaviour. With a noise-free pose, the first gesture is emitted after 267 ms at 30 FPS, exactly matching Eq. (5.6); a 60 FPS camera reduces this to 200 ms. Because the 50 ms cap discards every other frame of a 30 FPS stream, the effective inference rate is 15 FPS rather than 20 FPS. Holding the "OK" sign for 5 s produced exactly 5 clicks (at 0.27, 1.33, 2.33, 3.33 and 4.40 s), whereas without the 1 s cool-down the stabiliser would have emitted a click on each of the 72 stable frames. Finally, the classifier itself is very cheap: one million classifications took 144.5 ms on the development laptop, i.e. 0.145 µs per pose (about 6.9 million poses per second), so the cost of gesture recognition in the browser is dominated by MediaPipe's landmark model, not by GeoSwipe's own logic.

Table 7.6: Timing behaviour of the gesture subsystem (simulation on the development laptop)

Measure	Value
Time to first emission, 30 FPS camera, noise-free	267 ms (= 4 × 2 × 33.3 ms)
Time to first emission, 60 / 24 / 15 FPS camera	200 / 333 / 267 ms
Effective inference rate, 30 FPS camera	15.2 frames/s
Clicks emitted while holding "OK" for 5 s (with / without cool-down)	5 / 72
Classifier cost per pose	0.145 µs (≈ 6.9 × 10^6 poses/s)

7.2.6 Network Cost and Privacy of Gesture Recognition

The earlier server-side design streamed base64-encoded JPEG frames of about 30 KB at up to 30 FPS, i.e. roughly 900 KB/s or 3.2 GB per hour per active user, and its Python process did not fit into the 512 MB memory of the free hosting tier. In the final design, the only network cost of gestures is a one-time download of the MediaPipe WASM runtime and model (the model file is 7,819,105 bytes on the production site), after which the files are cached by the browser for 30 days. Figure 7.5 plots cumulative data transferred against minutes of gesture use: the in-browser design breaks even with the streaming design after about 11 s of use and saves about 3.2 GB per user-hour thereafter. Table 7.7 summarises the comparison; the qualitative gain is that video frames never leave the learner's device.

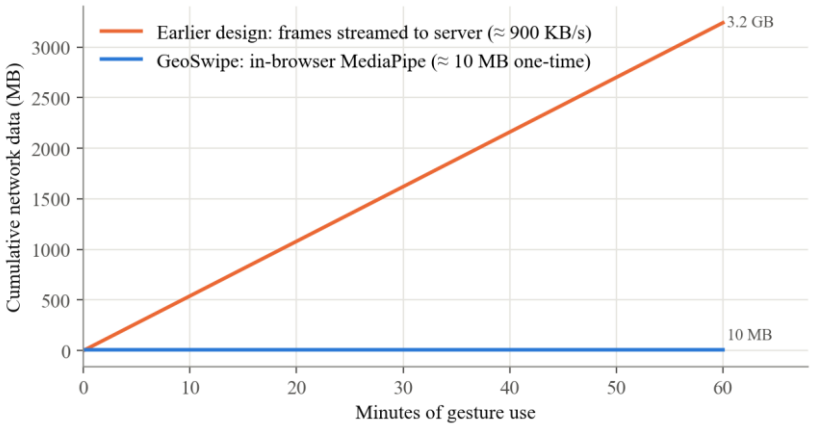


Figure 7.5: Cumulative network data for gesture recognition: server-side streaming versus in-browser inference

Table 7.7: Server-side versus in-browser gesture recognition

Aspect	Earlier: Python + OpenCV on server	Final: MediaPipe in browser
Network per active user	≈ 900 KB/s upstream (≈ 3.2 GB/h)	≈ 10 MB once, then cached
Server compute	Persistent Python process; exceeded 512 MB tier	None
Latency	Frame upload + inference + result download	Local inference only
Privacy	Webcam frames sent to server	Frames never leave the device
Behavioural fidelity	Reference implementation	0 mismatches on 20,000 poses (T3)

7.2.7 Front-End Bundle Size and Page Load

The production build consists of 38 JavaScript and CSS assets totalling 3.24 MiB (0.90 MiB after gzip level 9 compression, which we computed for comparison; the CDN may use Brotli). Thanks to route-level code splitting, the **initial** load of the landing page needs only three files — the entry chunk, the React vendor chunk and the main stylesheet — totalling 318 KiB raw and 95 KiB compressed. Figure 7.6 shows that the largest chunks are libraries loaded only on the pages that need them: MapLibre (918 KiB raw) for the heritage map, the trip-planner result chunk (529 KiB, which bundles jsPDF for PDF export) and Three.js (474 KiB) for the globe; the MediaPipe vision bundle is 140 KiB (42 KiB compressed).

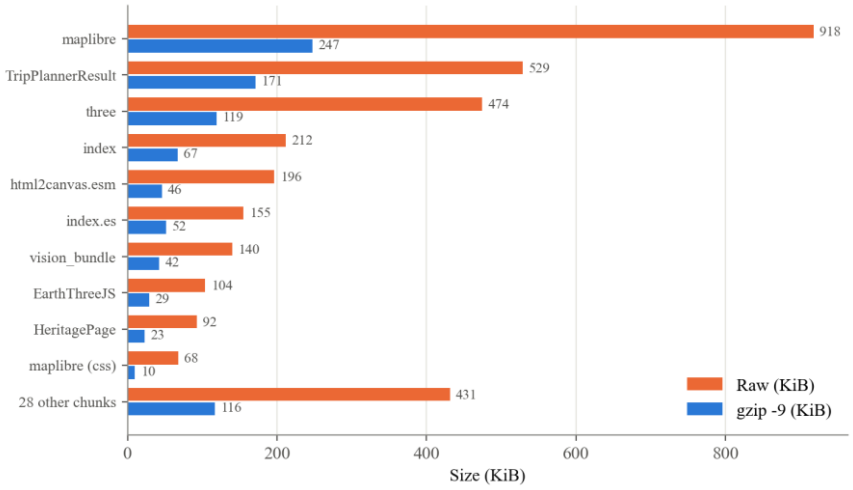


Figure 7.6: Size of the largest production chunks, raw and compressed

The First Contentful Paint (FCP) of each route was also recorded with headless Chrome during the screenshot run of Chapter 6, with GPU emulation in software. Most routes painted in 100–400 ms; the Explore page, which initialises the WebGL globe, took 1.56 s. Because software rendering is slower than a real GPU, these figures are conservative for the globe, and because each route was visited in sequence, they benefit from the browser cache after the first visit.

7.2.8 Content Coverage

Figure 7.7 summarises the heritage content. The 126 sites comprise 36 UNESCO World Heritage entries, 26 monuments, 26 temples, 18 historic forts, 7 rock-cut caves, 6 historic buildings, 5 palaces and 2 museums, located in 22 states and union territories, with Maharashtra (34) and Rajasthan (19) best represented. All 126 sites include access-route and visitor information; 22 include a 360° view. The quiz bank contains 780

questions for 13 monuments, equally split across three difficulty levels, and six monuments have five-chapter narrated storybooks.

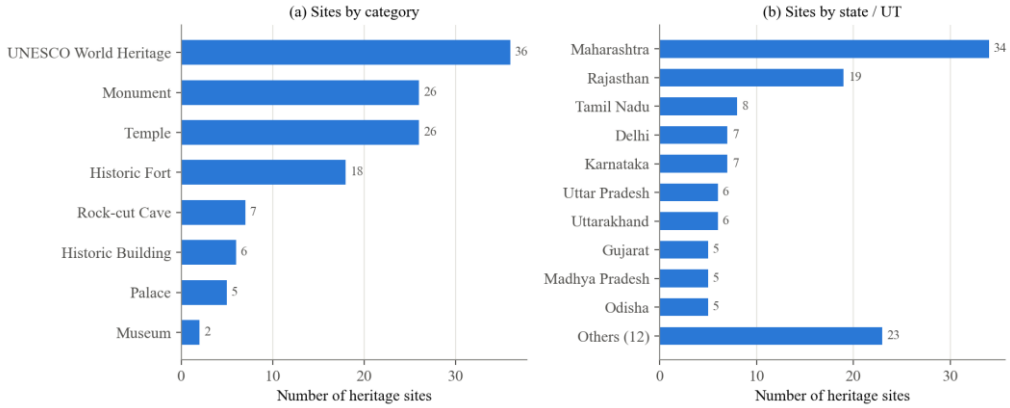


Figure 7.7: Distribution of heritage sites by category and by state or union territory

7.3 Screenshots

Screenshots of every module of the running system, captured from the production deployment, are presented and explained in Section 6.2 (Figure 6.1 to Figure 6.8); they are not repeated here.

7.4 Comparative Analysis

Table 7.8 compares GeoSwipe's capabilities with the systems and frameworks reviewed in Chapter 2. The comparison is qualitative, because the reviewed works target different goals and do not report a common quantitative metric; a ✓ indicates that the capability is a stated part of the work as described in its publication, "XR" denotes interaction through extended-reality devices, and "—" means the capability is not a stated part of the work.

Table 7.8: Capability comparison with systems from the literature survey

System	3D / globe	Heritage content	Games	Gestures	AI assistant	Trip / context	Multi-player	Browser, no install
Liritzis et al. [4]	✓	✓	—	—	—	—	—	—
Okanovic et al. [5] (XR)	✓	✓	—	XR	—	—	—	—
Belhi et al. [6]	✓	✓	—	—	—	—	—	—
Scianna, La Guardia [7]	✓	✓	—	—	—	—	—	✓
Wüest et al. [8]	✓	—	—	—	—	—	—	✓
Kölsch, Turk [10]	—	—	—	✓	—	—	—	—

System	3D / globe	Heritage content	Games	Gestures	AI assistant	Trip / context	Multi-player	Browser, no install
Chatbots reviewed in [12]	—	—	—	—	✓	—	—	✓
Fogli, Sansonetti [13]	—	—	—	—	—	✓	—	—
Wendel et al. [14]	—	—	✓	—	—	—	✓	—
GeoSwipe	✓	✓	✓	✓	✓	✓	✓	✓

The comparison highlights GeoSwipe's main contribution: integration. Each reviewed work excels in one or two dimensions — high-fidelity reconstruction of a single site [4], immersive XR interaction [5], professional geovisualisation [7], [8] or context-aware recommendation [13] — but none combines them in one workflow available in an ordinary browser. GeoSwipe does not match the visual fidelity of a dedicated reconstruction such as [4] or the immersion of XR [5]; it trades these for breadth of content (126 sites), zero installation, privacy-preserving gestures and zero operating cost. Against its own earlier version, the final design delivers measurable improvements, summarised in Table 7.9.

Table 7.9: Earlier prototype versus final GeoSwipe

Measure	Earlier prototype	Final system
API keys in browser bundle	Yes (Groq, OpenWeather, MapTiler)	None
NewsAPI calls, 4-view sequence	≈ 32	5
Weather response when cached	Not cached (live call)	4–6 ms server-side
Safe-places response when cached	8,726 ms	2 ms
Gesture network traffic	≈ 3.2 GB per user-hour	≈ 10 MB once
Gesture compute location	Server (Python)	User's browser
Integrations healthy on production	Deployment failed (memory)	9 / 10 (all required)

7.5 Discussion and Limitations

The results show that GeoSwipe is technically sound and operable at zero cost: caching turns third-party quotas from a per-user limit into a per-site limit, the API responds within a few hundred milliseconds when warm, and the gesture subsystem converts noisy per-frame classifications into reliable actions. Several limitations qualify these findings. First, the gesture results are from a *synthetic* benchmark with Gaussian

landmark noise and idealised templates; real hands vary in shape, pose and lighting, MediaPipe's landmark errors are not independent Gaussian, and the benchmark does not measure MediaPipe itself. A study with real participants and a physical webcam (test T13) is required. Second, end-to-end latency depends on the client's network and on the free hosting tier; the 23.1 s cold start after inactivity is noticeable, and a keep-warm ping or a paid instance would remove it. Third, news relevance depends on NewsAPI's keyword search; during screenshot capture we observed that a general query can return articles unrelated to heritage, which suggests filtering results by domain or switching to a curated source. Finally, we have not yet measured usability or learning gains; the System Usability Scale [15], [18] and pre- and post-tests are planned (Chapter 8).

Chapter 8: Conclusion and Future Scope

8.1 Conclusion

We set out to build a single browser-based platform that connects 3D geographic exploration with Indian cultural heritage, game-based and multiplayer learning, AI-assisted guidance and touch-free gesture interaction, while protecting privacy and remaining deployable at no cost. GeoSwipe meets each of the objectives stated in Section 1.3. Learners can rotate and click an adaptive WebGL globe; explore 126 heritage sites in eight categories on a MapLibre map with detailed information, access routes, visitor information, images, 360° views, 3D models, live weather and news; read six narrated storybooks; play country, flag and heritage quizzes drawn from 780 curated questions, alone or against a classmate in synchronised ten-round Socket.IO games; ask an LLM-based heritage assistant; generate day-wise itineraries with rupee cost estimates and PDF export; and rank routes with a transparent safety score. All of this is available without installation or an account, and every gesture action has a mouse or touch equivalent.

The engineering contributions are as important as the features. Moving gesture recognition from a server-side Python process into the browser eliminated about 3.2 GB of upstream traffic per user-hour, removed all server compute for gestures and ensured that webcam frames never leave the learner's device, while a differential test on 20,000 poses showed that behaviour was preserved exactly. In a synthetic robustness benchmark the classifier remained at least 99.06 % accurate up to a landmark noise of $\sigma = 0.01$, and the five-frame stabiliser emitted no wrong gestures in any tested condition, at the cost of added delay under heavy noise. A shared two-tier cache with request de-duplication, stale-on-failure serving and key pooling made third-party usage scale with the number of sites instead of the number of users: warm responses were 20× to 4,363× faster than cold ones and a representative news sequence used 5 instead of about 32 NewsAPI calls. The hardened deployment keeps every API key on the server, passes 9 of its 10 integration health checks (all required ones) and answers warm API requests with medians of 351–2,233 ms from a client in Mumbai. Together, these results demonstrate that an integrated, privacy-preserving geographic and heritage learning platform is technically and economically feasible for a student team using free

infrastructure. Whether it improves learning outcomes has not yet been measured and is the most important open question.

8.2 Future Scope

1. **User evaluation.** Conduct a controlled study with students measuring perceived usability with the System Usability Scale [15], [18], learning gains with pre- and post-tests, and engagement from interaction logs, comparing GeoSwipe with conventional material.
2. **Real-world gesture validation.** Record gesture accuracy, latency and fatigue with real users and webcams under varied lighting, and replace fixed distance thresholds with thresholds normalised by hand size, or with a small learned classifier trained on MediaPipe landmarks.
3. **Mobile and offline support.** Package GeoSwipe as a Progressive Web App with offline storybooks and quizzes, and adapt gesture controls and layouts for phones and tablets.
4. **Content expansion.** Add storybooks and quiz banks for all 126 sites, extend coverage to under-represented states, add regional-language narration and captions, and open a contributor workflow for teachers.
5. **Learner accounts and analytics.** Optional accounts to track progress, recommend sites and quizzes adaptively, and give teachers class dashboards.
6. **Classroom multiplayer.** Extend rooms beyond two players (for example, whole-class quiz sessions), moving room state to a shared store so that the server can scale horizontally.
7. **Better context services.** Filter news by relevance and trusted sources, cache the full site catalogue response, and keep the free back end warm to remove cold starts.
8. **Richer AI grounding.** Ground the assistant in each site's full stored description with retrieval, cite the source records in answers, and evaluate answer accuracy systematically.

References

[1] Z. Yu, M. Gao, and L. Wang, “The effect of educational games on learning outcomes, student motivation, engagement and satisfaction,” *J. Educ. Comput. Res.*, vol. 59, no. 3, pp. 522–546, 2021.

[2] A. I. Abdul Jabbar and P. Felicia, “Gameplay engagement and learning in game-based learning: A systematic review,” *Rev. Educ. Res.*, vol. 85, no. 4, pp. 740–779, 2015.

[3] M. Mortara and C. E. Catalano, “3D virtual environments as effective learning contexts for cultural heritage,” *Italian J. Educ. Technol.*, 2018.

[4] I. Liritzis, P. Volonakis, and S. Vosinakis, “3D reconstruction of cultural heritage sites as an educational approach: The Sanctuary of Delphi,” *Appl. Sci.*, vol. 11, no. 8, Art. no. 3635, 2021.

[5] V. Okanovic, I. Ivkovic-Kihic, D. Boskovic, B. Mijatovic, I. Prazina, E. Skaljo, and S. Rizvic, “Interaction in eXtended Reality applications for cultural heritage,” *Appl. Sci.*, vol. 12, no. 3, Art. no. 1241, 2022.

[6] A. Belhi, H. O. Ahmed, T. Alfaqheri, A. Bouras, A. H. Sadka, and S. Foufou, “An integrated framework for the interaction and 3D visualization of cultural heritage,” *Multimedia Tools Appl.*, vol. 83, no. 15, 2024.

[7] A. Scianna and M. La Guardia, “Globe based 3D GIS solutions for virtual heritage,” *Int. Arch. Photogramm. Remote Sens. Spatial Inf. Sci.*, vol. XLII-4/W10, pp. 171–177, 2018.

[8] R. Wüest, H. Eugster, S. Nebiker, and M. Christen, “Processing and rendering massive 3D geospatial environments using WebGL — The examples of OpenWebGlobe and SmartMobileMapping,” in *Proc. Graphical Web*, 2012.

[9] X. Zabulis, H. Baltzakis, and A. A. Argyros, “Vision-based hand gesture recognition for human–computer interaction,” in *The Universal Access Handbook*, C. Stephanidis, Ed. Boca Raton, FL, USA: CRC Press, 2009.

[10] M. Kölsch and M. Turk, “Vision-based hand gesture interfaces for wearable computing and virtual environments,” 2004.

[11] B. Dong, J. Bai, T. Xu, and Y. Zhou, “Large language models in education: A systematic review,” in *Proc. 6th Int. Conf. Comput. Sci. Technol. Educ. (CSTE)*, 2024.

[12] H.-A. Bekkar and Y. Chtouki, “Chatbots in education: A systematic literature review,” in *Proc. 10th Int. Conf. Smart Comput. Commun. (ICSCC)*, 2024, pp. 637–644.

[13] A. Fogli and G. Sansonetti, “Exploiting semantics for context-aware itinerary recommendation,” *Pers. Ubiquitous Comput.*, vol. 23, no. 6, 2019.

[14] V. Wendel, F. Hertin, S. Göbel, and R. Steinmetz, “Collaborative learning by means of multiplayer serious games,” in *Advances in Web-Based Learning — ICWL 2010*, 2010.

[15] K. Orfanou, N. Tselios, and C. Katsanos, “Perceived usability evaluation of learning management systems: Empirical evaluation of the System Usability Scale,” *Int. Rev. Res. Open Distrib. Learn.*, vol. 16, no. 2, pp. 227–246, 2015.

[16] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, “MediaPipe Hands: On-device real-time hand tracking,” arXiv:2006.10214, 2020.

[17] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, “MediaPipe: A framework for building perception pipelines,” arXiv:1906.08172, 2019.

[18] J. Brooke, “SUS: A ‘quick and dirty’ usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

[19] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1981.

[20] I. Fette and A. Melnikov, “The WebSocket protocol,” IETF RFC 6455, Dec. 2011.

[21] R. W. Sinnott, “Virtues of the haversine,” *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

Published Papers

[INSERT: Copy of the published / submitted research paper — "GeoSwipe: Interactive Earth Exploration and Indian Heritage Learning using Hand Gestures" — with the conference or journal name, date and acceptance proof.]

Certificates

[INSERT: Participation / presentation / publication certificates of all four students (e.g., conference, Mapathon or project competition certificates).]

Plagiarism Report

[INSERT: Plagiarism report generated by the institute-approved tool (tool name, date, similarity index), signed by the project guide.]